

DOCUMENT 5

Implementation of FFT IP and DMA IP with Softcore MicroBlaze processor and Hardcore Zynq Processor

KAPISH GUPTA

IP (Intellectual Property)

In FPGA design, an IP (Intellectual Property) is simply a readymade hardware block. Instead of writing complex Verilog or VHDL code from scratch, we can just drag and drop these pre-tested blocks into our design.

FFT IP

This is a readymade block provided by Xilinx. It takes the data and calculates the Fast Fourier Transform mathematically using physical logic gates, which is much faster than running C code.

Implementation of FFT IP and DMA IP with Softcore MicroBlaze processor

Here, we will select the transform length to be 8.

Fast Fourier Transform (9.1)

Documentation IP Location

IP Symbol Implementation Details Latency

Information

Implementation: Pipelined, Streaming I/O

Transform Size

Largest:	8
Smallest:	8
Output Data Width:	32

Resource Estimates Group

DSP48 Slices:	14
Block RAMs:	2

AXI4 Stream Port Structure

S_AXIS_DATA - TDATA

Transaction	Field	Type
0	CHAN_0_XN_IM_0(63:32)	float_single
	CHAN_0_XN_RE_0(31:0)	float_single
1	CHAN_0_XN_IM_1(63:32)	float_single
	CHAN_0_XN_RE_1(31:0)	float_single
2	CHAN_0_XN_IM_2(63:32)	float_single
	CHAN_0_XN_RE_2(31:0)	float_single
3	CHAN_0_XN_IM_3(63:32)	float_single
	CHAN_0_XN_RE_3(31:0)	float_single
...		
7	CHAN_0_XN_IM_7(63:32)	float_single
	CHAN_0_XN_RE_7(31:0)	float_single

S_AXIS_CONFIG - TDATA

Component Name: xfft_0

Configuration Implementation Detailed Implementation

Number of Channels: 1

Transform Length: 8

Architecture Configuration

Target Clock Frequency (MHz): 250 [1 - 1000]

Target Data Throughput (MSPS): 50 [1 - 1000]

Architecture Choice

☒ Automatically Select

☐ Pipelined, Streaming I/O

☐ Radix-2, Burst I/O

☐ Radix-2 Lite, Burst I/O

☐ Run Time Configurable Transform Length

In addition, we will select Data format as Floating Point

And, output ordering as Natural order.

Fast Fourier Transform (9.1)

Documentation IP Location

IP Symbol Implementation Details Latency

AXI4 Stream Port Structure

S_AXIS_DATA - TDATA

Transaction	Field	Type
0	CHAN_0_XN_IM_0(63:32)	float_single
	CHAN_0_XN_RE_0(31:0)	float_single
1	CHAN_0_XN_IM_1(63:32)	float_single
	CHAN_0_XN_RE_1(31:0)	float_single
2	CHAN_0_XN_IM_2(63:32)	float_single
	CHAN_0_XN_RE_2(31:0)	float_single
3	CHAN_0_XN_IM_3(63:32)	float_single
	CHAN_0_XN_RE_3(31:0)	float_single
...		
7	CHAN_0_XN_IM_7(63:32)	float_single
	CHAN_0_XN_RE_7(31:0)	float_single

S_AXIS_CONFIG - TDATA

Transaction	Field	Type
0	SCALE_SCH_0(4:1)	bit4
	FWD_INV_0(0:0)	bit1

M_AXIS_DATA - TDATA

Transaction	Field	Type
0	CHAN_0_XN_IM_0(63:32)	float_single
	CHAN_0_XN_RE_0(31:0)	float_single
1	CHAN_0_XN_IM_1(63:32)	float_single
	CHAN_0_XN_RE_1(31:0)	float_single
2	CHAN_0_XN_IM_2(63:32)	float_single
	CHAN_0_XN_RE_2(31:0)	float_single

Component Name: xfft_0

Configuration Implementation Detailed Implementation

MANUAL Data Format: Floating Point

Scaling Options: Scaled

Rounding Modes: Truncation

Precision Options

Input Data Width: 32

Phase Factor Width: ☒ 24 ☐ 25

Control Signals

☐ ACLKEN ☒ ARESETn (active low)

ARESETn must be asserted for a minimum of 2 cycles

Output Ordering Options

Output Ordering: Natural Order

☐ Cyclic Prefix Insertion

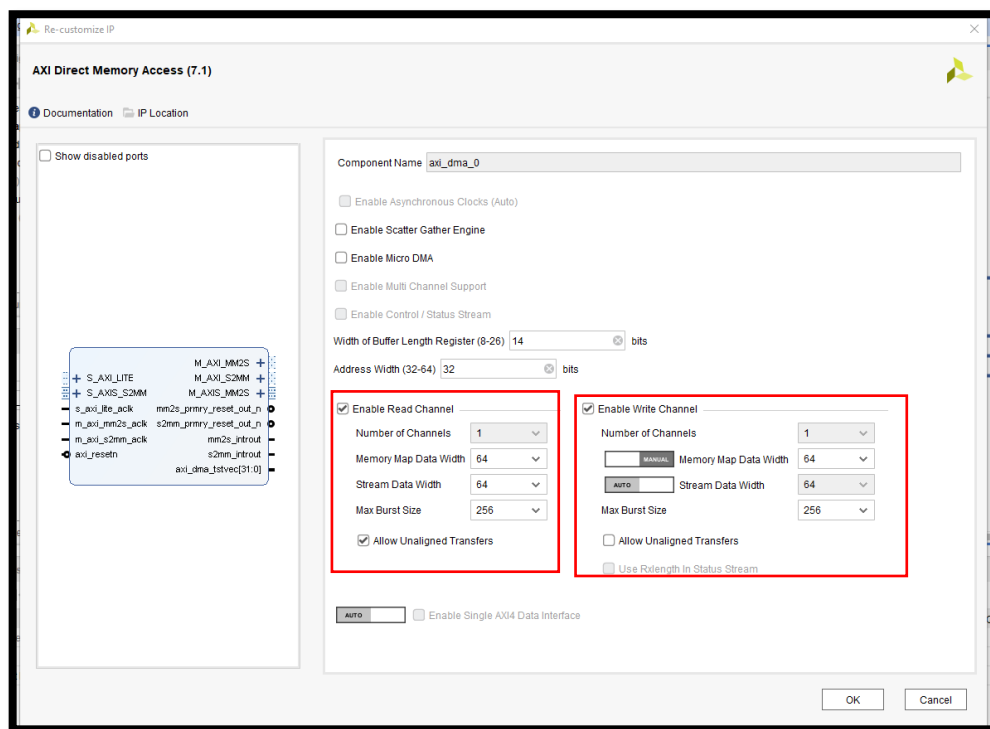
Optional Output Fields: ☐ XK_INDEX ☐ OVFO

Throttle Scheme: ☒ Non Real Time ☐ Real Time

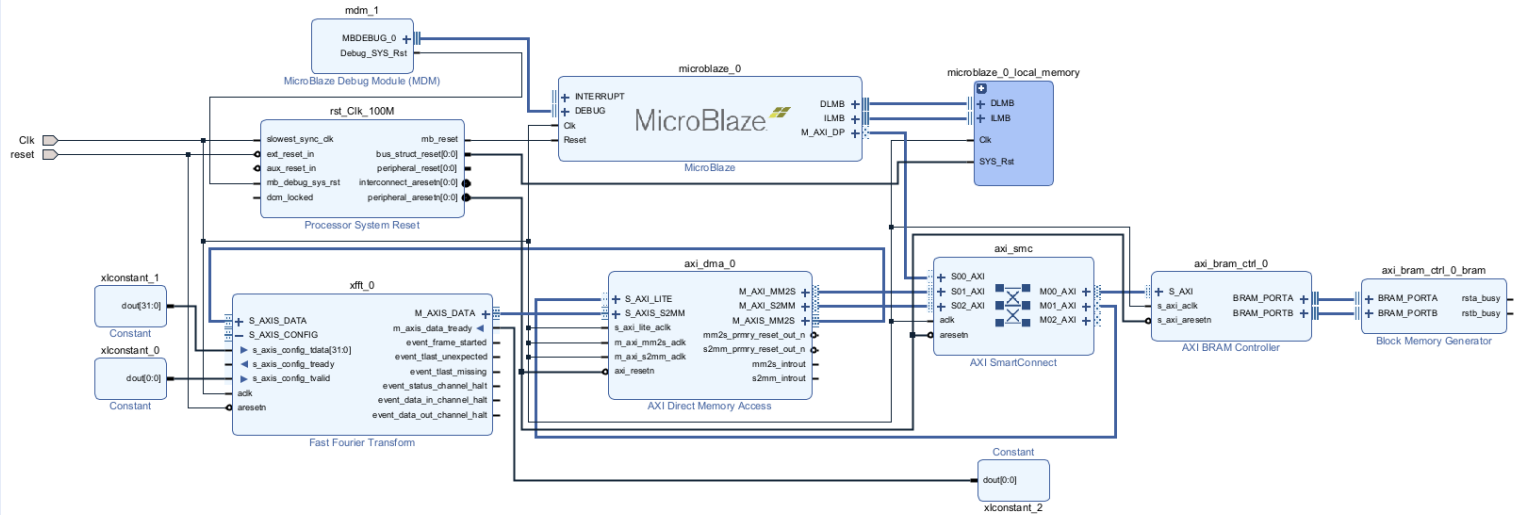
DMA

Direct Memory Access (DMA) is a hardware block that allows data to move directly between the memory and IO devices without the intervention of CPU.

The best part about DMA is that it does this without keeping the main processor busy, saving a lot of time.



Block Diagram



C Code

1. Libraries Included

```

1 #include <stdio.h>
2 #include "xil_printf.h"
3 #include "xparameters.h"
4 #include "xaxidma.h"
5 #include "xio.h"
6 #include "xil_cache.h"
7
8 #define FFT_Size 512
9 #define BRAM_OFFSET 0X8000
10 #define input_sel 1
11
12 #define fs 100000000.0
13 #define f 1000000.0
14
15 #ifndef M_PI
16 #define M_PI 3.14159265358979323846
17 #endif
18 //

```

2. Main Function

```
int main()
{
    // First real then imag : Each pair of data is a pair of real and imaginary.
    const float FFT_input[2*N] = {
        0x41300000,0x41b80000,0x42000000,0x41200000,0x42b60000,0x42bc0000,
        0x41700000,0x428a0000,0x423c0000,0x42c00000,0x42300000,0x41400000,
        0x42c00000,0x41880000,0x42440000,0x00000000
    };

    int status;
    int init_addr = XPAR_BRAM_0_BASEADDR + BRAM_OFFSET;

    // Writing data into the BRAM
    for(int i=0;i<2*N;i++){
        XIo_Out32(init_addr,FFT_input[i]);
        init_addr = init_addr + 4; //Incrementing the address by 4 bytes since we are writing 32 bits data.
    }

    Xil_DCacheFlushRange((UINTPTR)(XPAR_BRAM_0_BASEADDR + BRAM_OFFSET), sizeof(float)*N*2 );

    XAxiDma my_dma;
    XAxiDma_Config *my_dma_config;

    //Copying of the configuration information given the base address of the DMA
    my_dma_config = XAxiDma_LookupConfigBaseAddr(XPAR_AXI_DMA_0_BASEADDR);

    //Configuration initialization
    XAxiDma_CfgInitialize(&my_dma,my_dma_config);
    // fft IP to DMA transfer configuration
    XAxiDma_SimpleTransfer(&my_dma,XPAR_BRAM_0_BASEADDR + BRAM_OFFSET+sizeof(float)*N*2,sizeof(float)*N*2,XAXIDMA_DEVICE_TO_DMA);
    // DMA to fft IP transfer configuration
    XAxiDma_SimpleTransfer(&my_dma,XPAR_BRAM_0_BASEADDR + BRAM_OFFSET,sizeof(float)*N*2,XAXIDMA_DMA_TO_DEVICE);

    // Checking whether the DMA has completed the transfers or not
    status = XAxiDma_ReadReg(XPAR_AXI_DMA_0_BASEADDR,0x4) & XAXIDMA_HALTED_MASK ;
    while(status!=1){
        status = XAxiDma_ReadReg(XPAR_AXI_DMA_0_BASEADDR,0x4) & XAXIDMA_HALTED_MASK ;
    }

    status = XAxiDma_ReadReg(XPAR_AXI_DMA_0_BASEADDR,0x34) & XAXIDMA_HALTED_MASK ;
    while(status!=1){
        status = XAxiDma_ReadReg(XPAR_AXI_DMA_0_BASEADDR,0x34) & XAXIDMA_HALTED_MASK ;
    }

    return 0 ;
}
```

1. Transfer Input values to BRAM

This code uses **hardcoded raw hexadecimal values**.

These hex numbers are just standard 32-bit floating-point numbers written in machine format.

There are 16 numbers total. Since an FFT needs a Real and an Imaginary part for every point, 16 numbers = **8 complex data points**.

It takes the first hex number, and stores it in the BRAM using the XIo_Out32 command.

2. Flush the Cache memory

Xil_DCacheFlushRange forces the processor to take the data sitting in its temporary, internal fast memory (Cache) and permanently push it out to the BRAM so the DMA hardware can actually see it and fetch the correct data instead of garbage.

3. The Setup (Initialization)

MicroBlaze wakes up and needs to find the DMA hardware. It uses the XPAR_AXIDMA_0_DEVICE_ID from the board support package to locate the DMA block. It runs XAxiDma_CfgInitialize to turn the DMA ON.

3. SimpleTransfer

The MicroBlaze uses function to start the hardware: XAxiDma_SimpleTransfer() It calls this twice. First, it tells the DMA where to put the incoming answers (S2MM) (stream to memory map). Then, it tells the DMA where to fetch the input data to send to the FFT (MM2S) (memory map to stream). The moment this second command runs, the DMA starts pulling the data out of the BRAM.

Transmit (DMA to FFT): XPAR_BRAM_0_BASEADDR + BRAM_OFFSET The DMA is told to start reading data from the very beginning of the BRAM (exactly where the for loop just finished writing the input data).

Receive (FFT to DMA): XPAR_BRAM_0_BASEADDR + BRAM_OFFSET + sizeof(float)*N*2 The DMA is told to write the answers back into the exact same BRAM, but **offset by 64 bytes** (which is exactly the size of the 16 input numbers).

In simple terms: If the input data takes up slots 0 through 15 in the BRAM, the DMA is told to save the FFT answers in slots 16 through 31. This prevents the output answers from overwriting and destroying the input data.

4. Wait for the transfer to complete

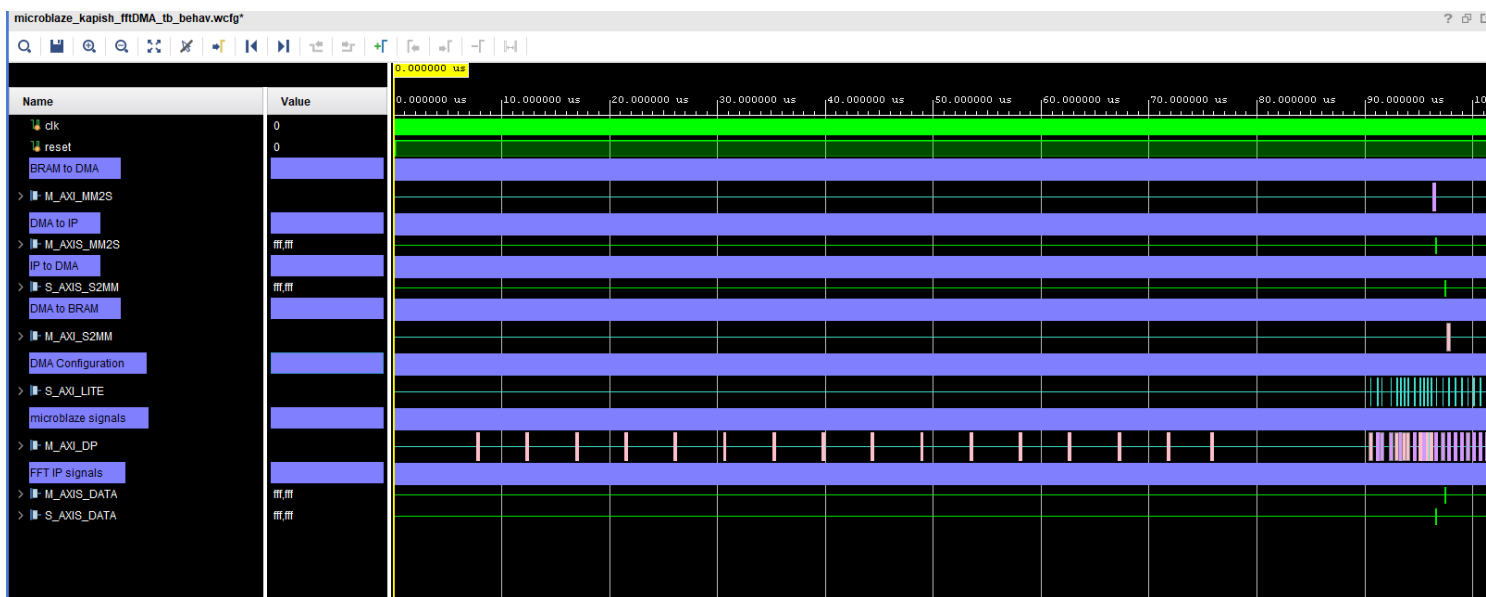
Because it is a simulation, you likely used a simple polling loop to wait for the transfer to finish. The MicroBlaze constantly checks the DMA's Status Register (checking if the HALTED bit turns to 1). Once the FFT IP finishes the math and flashes, the TLAST signal, the DMA stops, updates its register, and the MicroBlaze breaks out of the waiting loop.

After the C code, associate the **elf file** in the Vivado and run simulation.

MicroBlaze Simulation Results

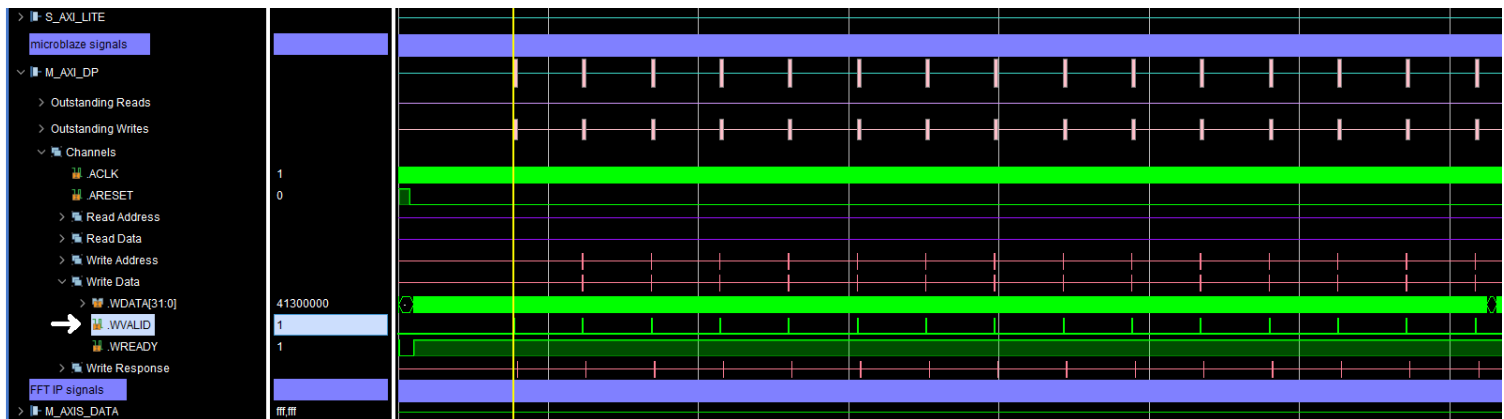
For the MicroBlaze block design, we chose a small FFT transform length of N=8 to easily observe the signals in the simulation. Here is how the data flows step-by-step:

Simulation:-

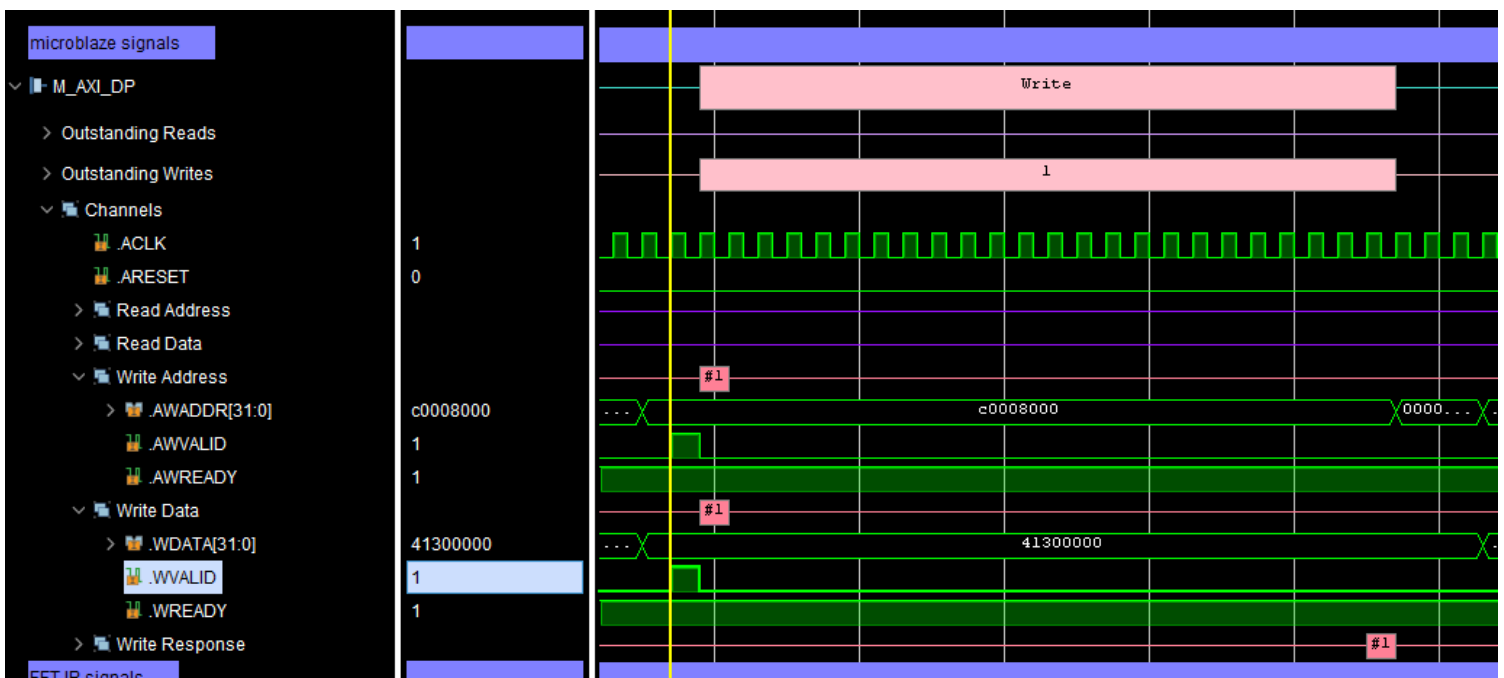


1. MicroBlaze to BRAM Write: First, the MicroBlaze processor writes our test data into the BRAM. Our processor uses a 32-bit data bus. Because one complex number consists of a 32-bit real part and a 32-bit imaginary part, the processor has to write twice for every single complex number. To store 8 complex numbers, it takes exactly 16 writes. In the simulation screenshot below, you can clearly count 16 `w_valid` (write valid) signals.

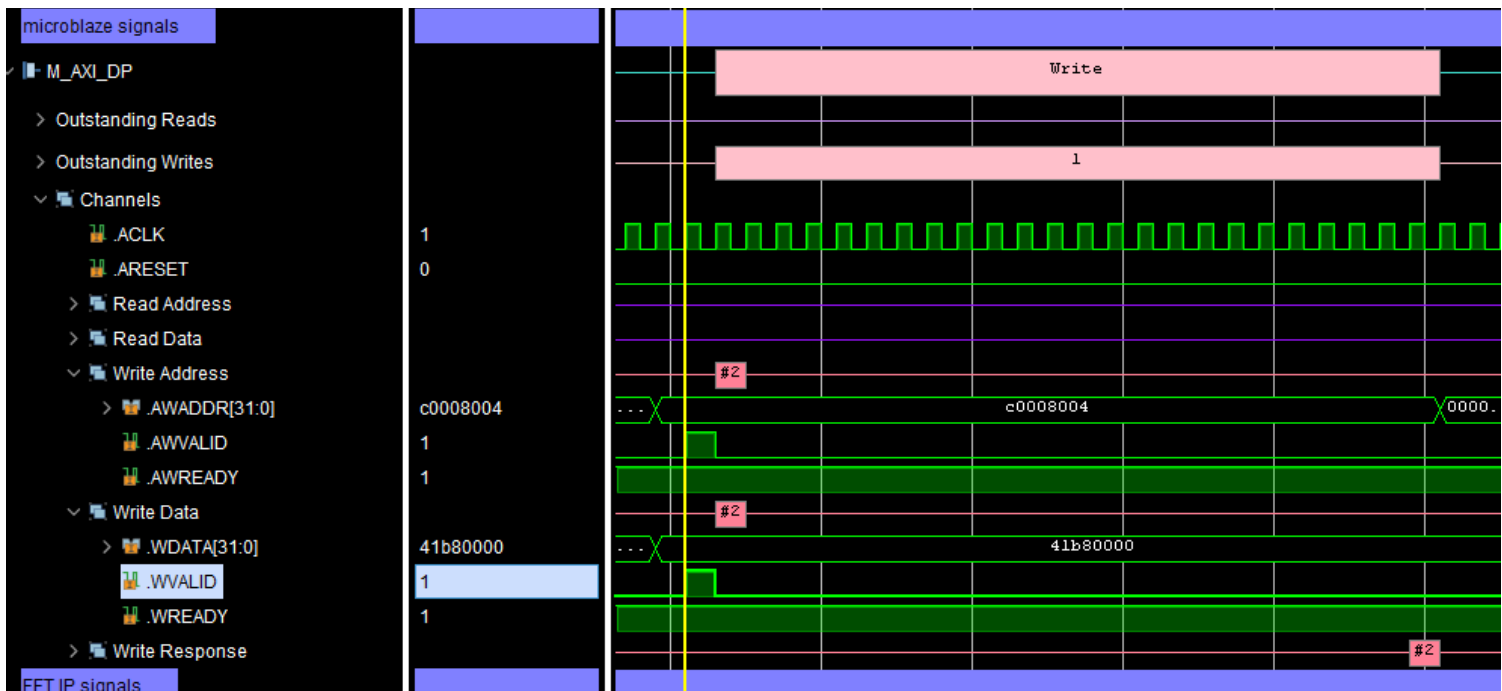
Here, we can see there are 16 write operations from MicroBlaze to BRAM.



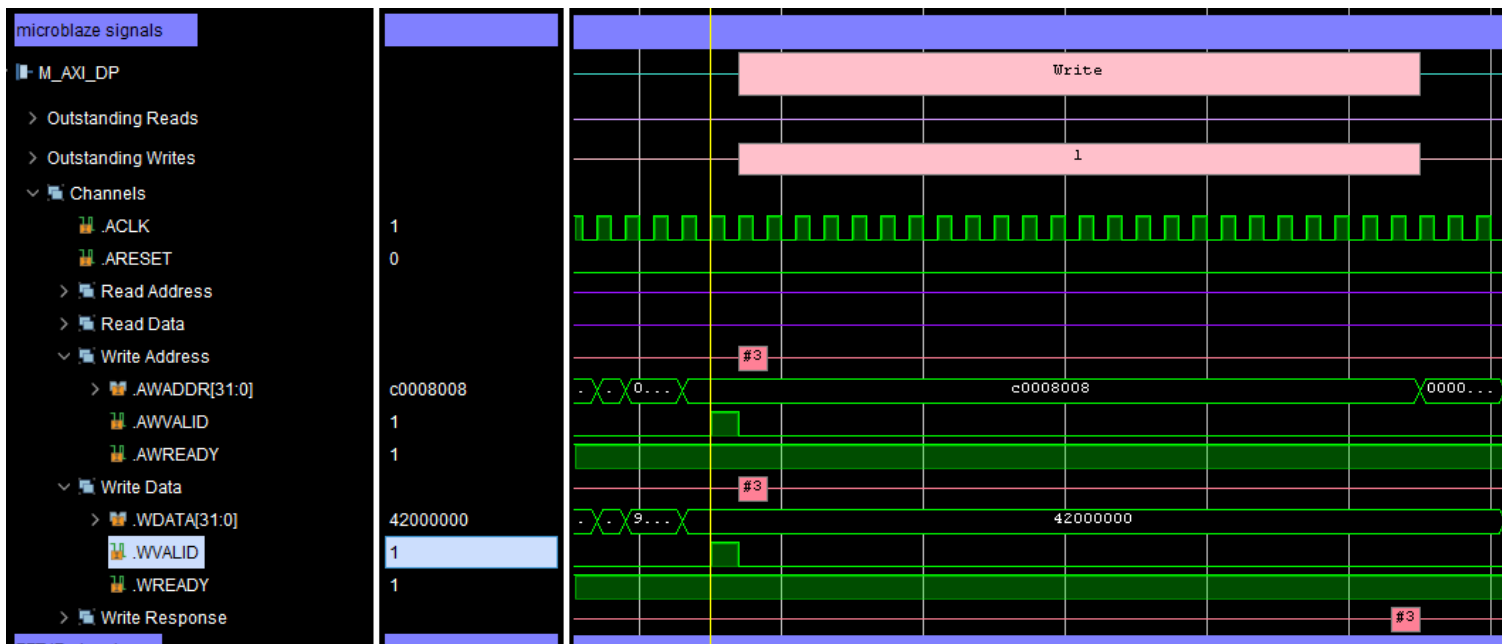
1st data is written at correct address location of BRAM



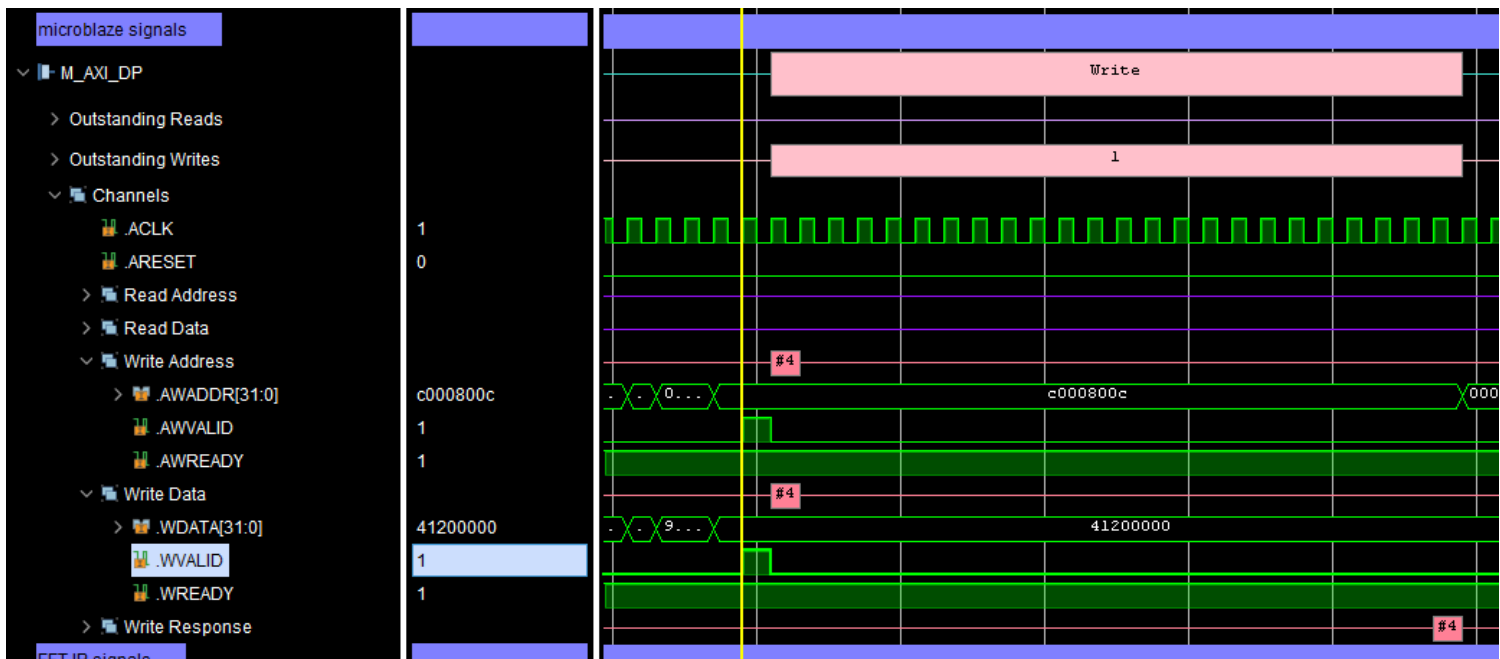
2nd data is written at correct address location of BRAM



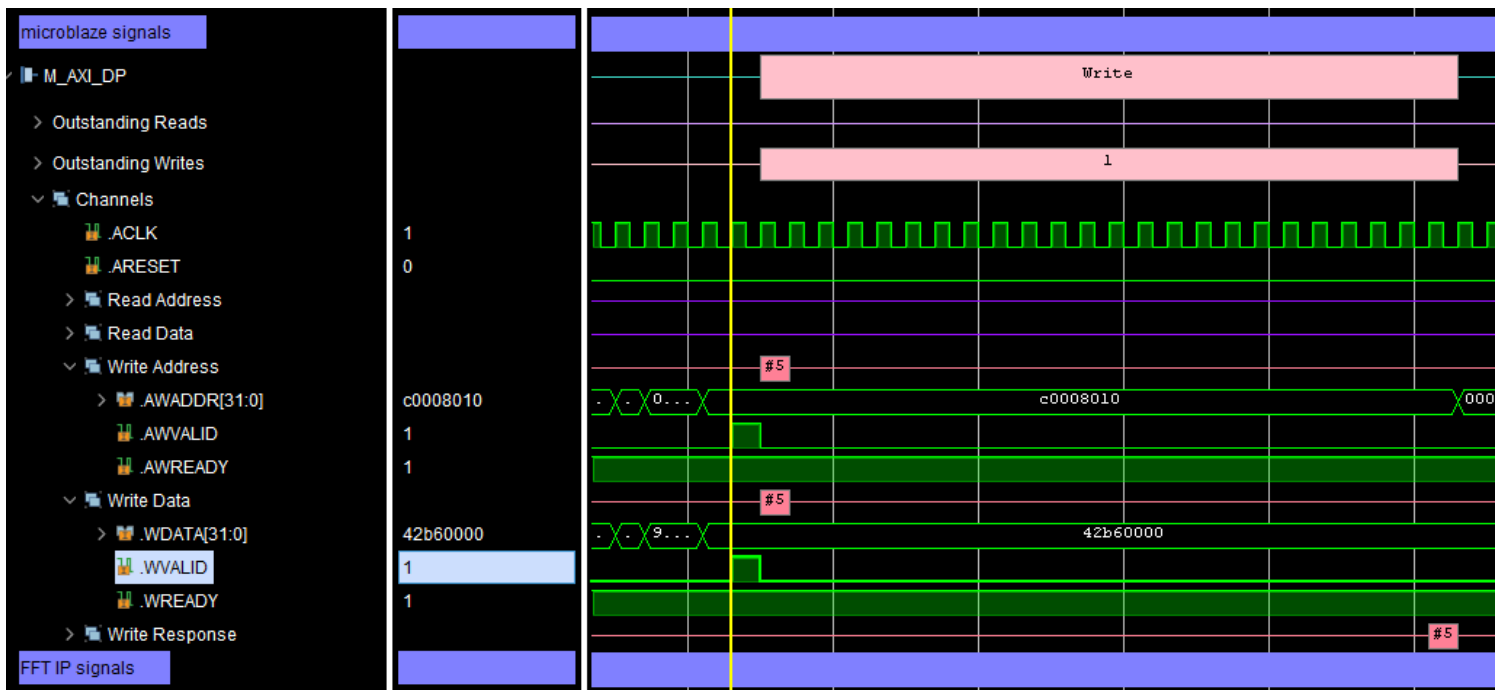
3rd data is written at correct address location of BRAM



4th data is written at correct address location of BRAM



5th data is written at correct address location of BRAM



Similarly, all the 16 data is written successfully

Input Data Flow :-

BRAM to DMA

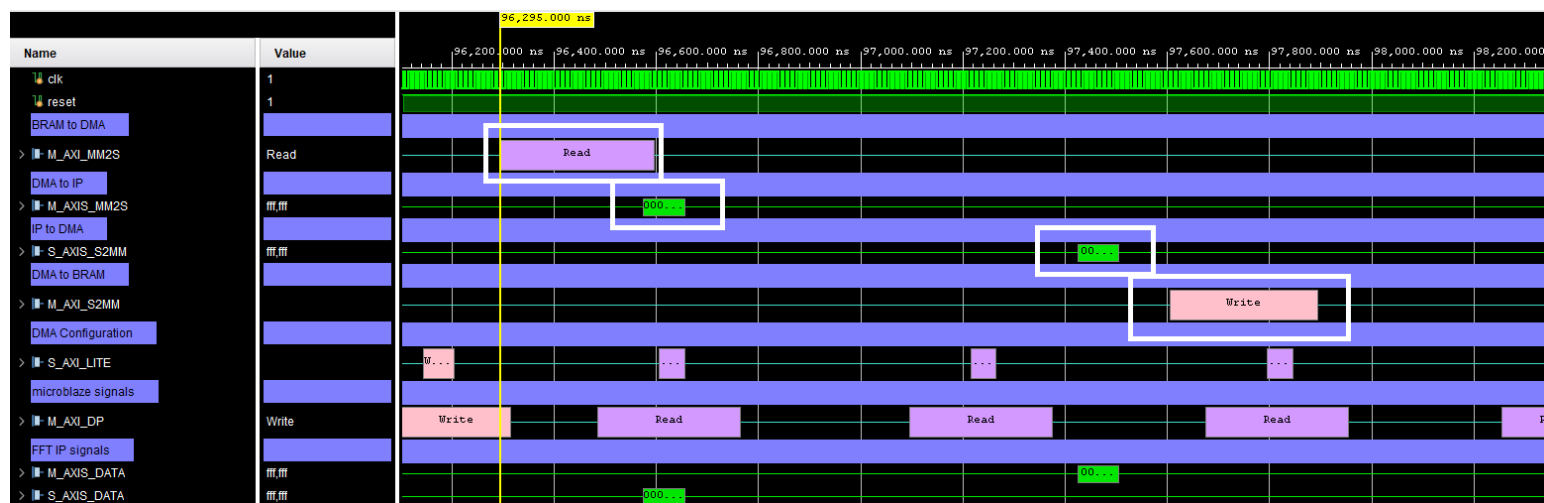
DMA to FFT IP

Output Data Flow :-

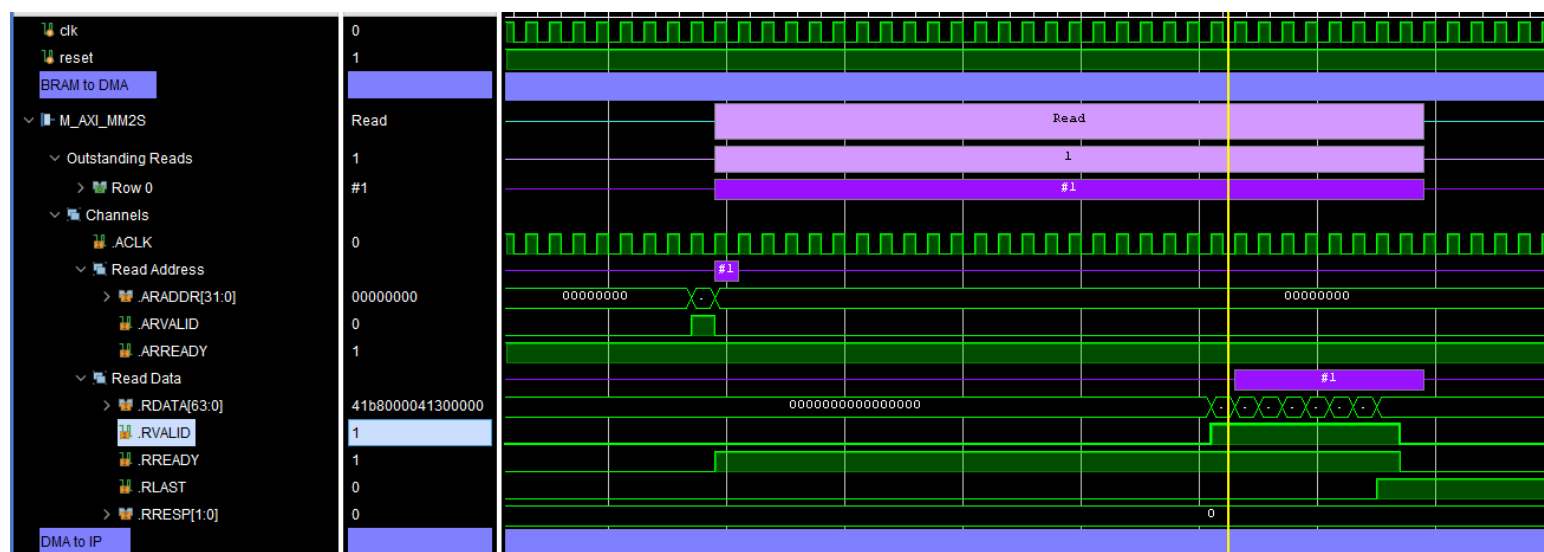
FFT IP to DMA

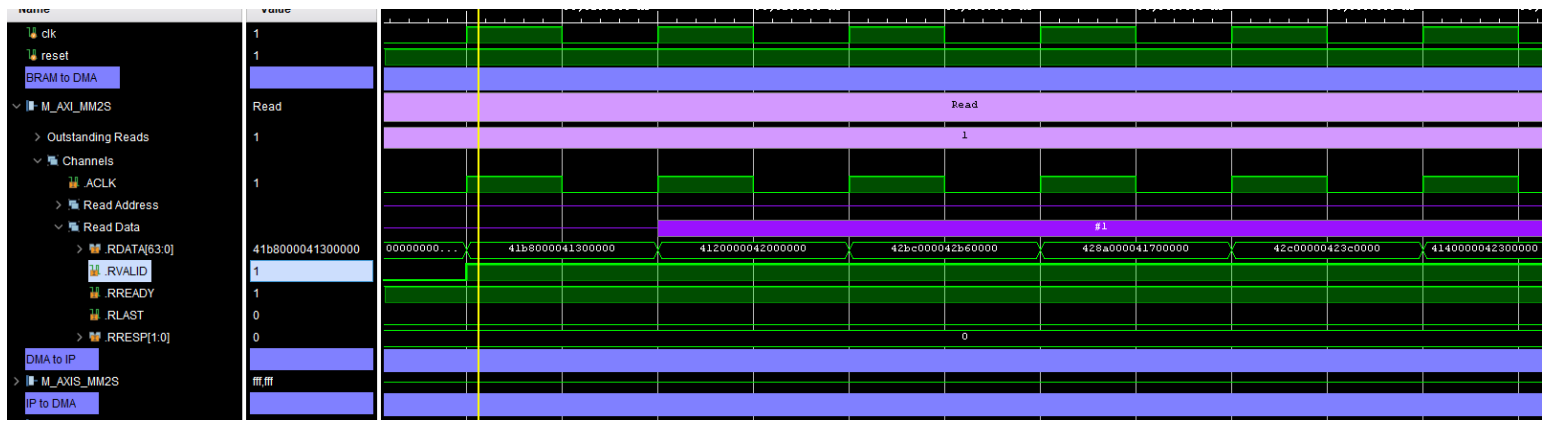
DMA to BRAM

We can see the simulation same as the data flow.

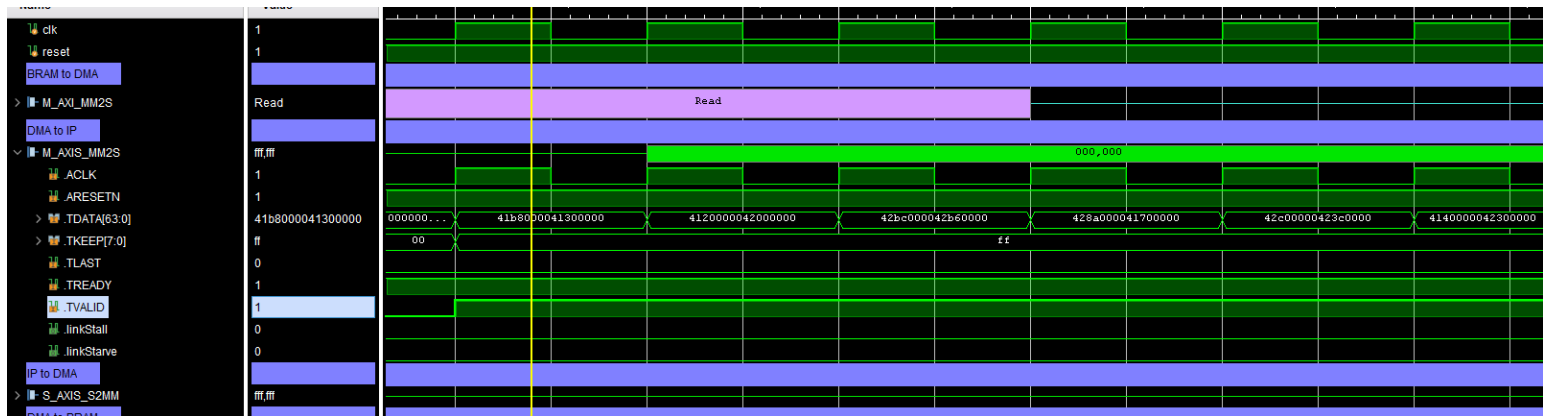


2. BRAM to DMA Transfer: Next, the data moves from the BRAM to the DMA. We configured the DMA to handle 64-bit data. Because of this, the DMA can grab both the 32-bit real part and the 32-bit imaginary part at the exact same time. Therefore, for 8 complex numbers, the simulation only shows 8 `tvalid` signals.

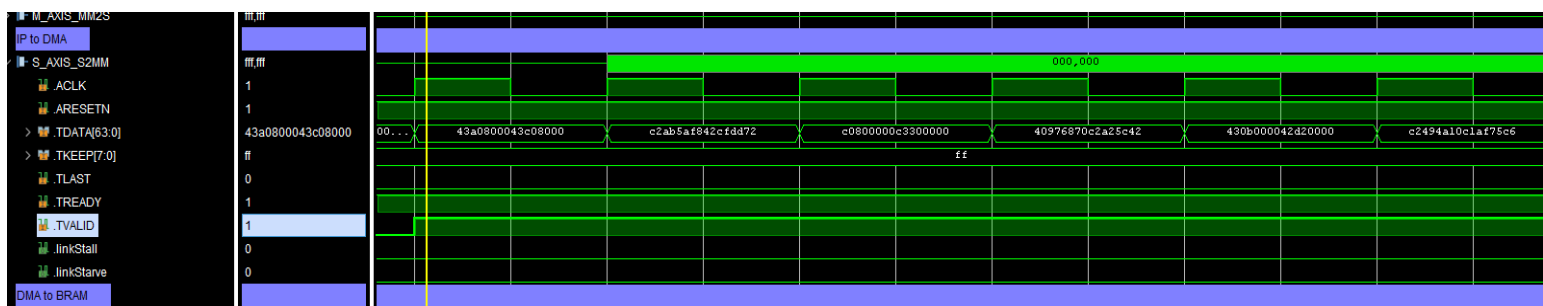




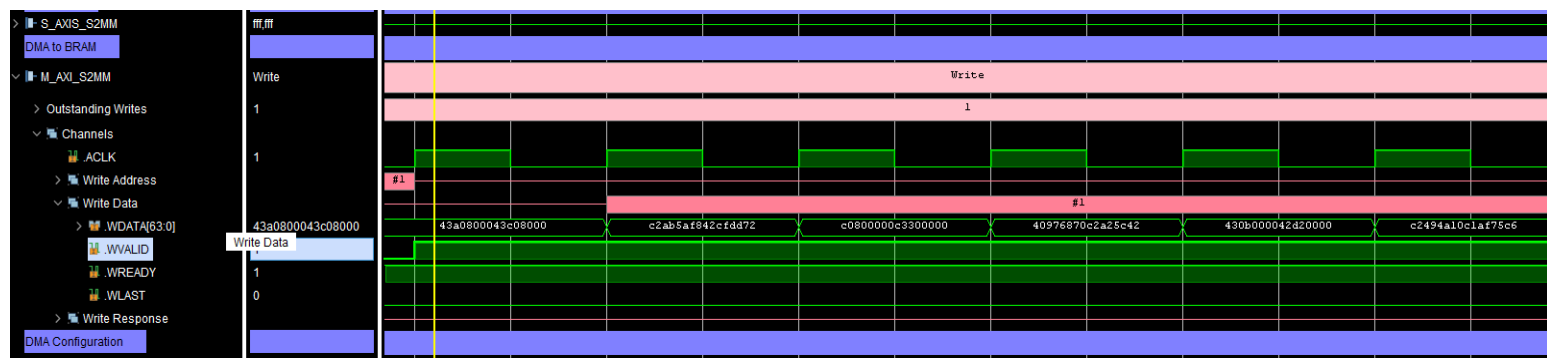
3. DMA to FFT: After that, the DMA streams those 8 packets into the FFT IP.



3. FFT to DMA: The FFT IP processes the signals and outputs the frequency data back to the DMA.

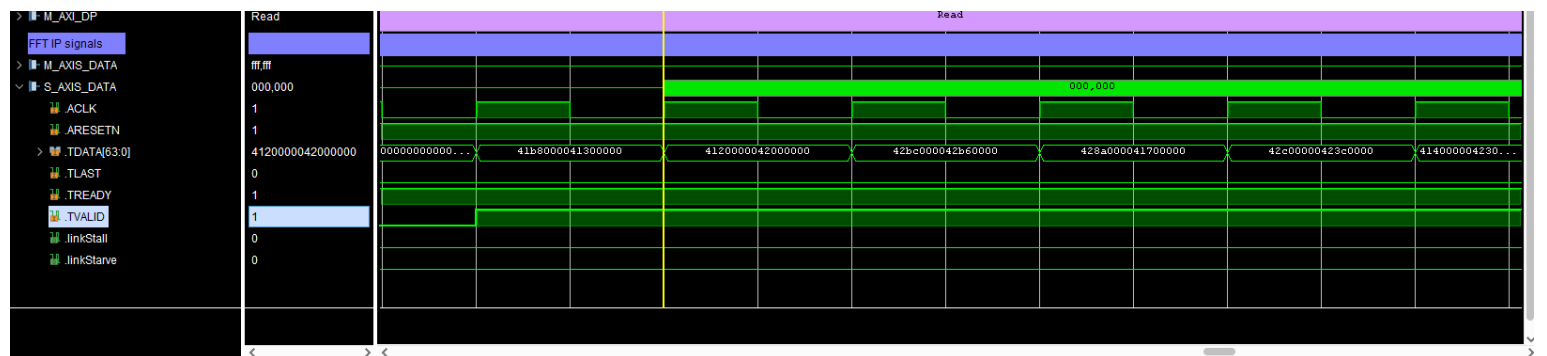


4. DMA to BRAM: Finally, the DMA writes the final output data back into the BRAM.

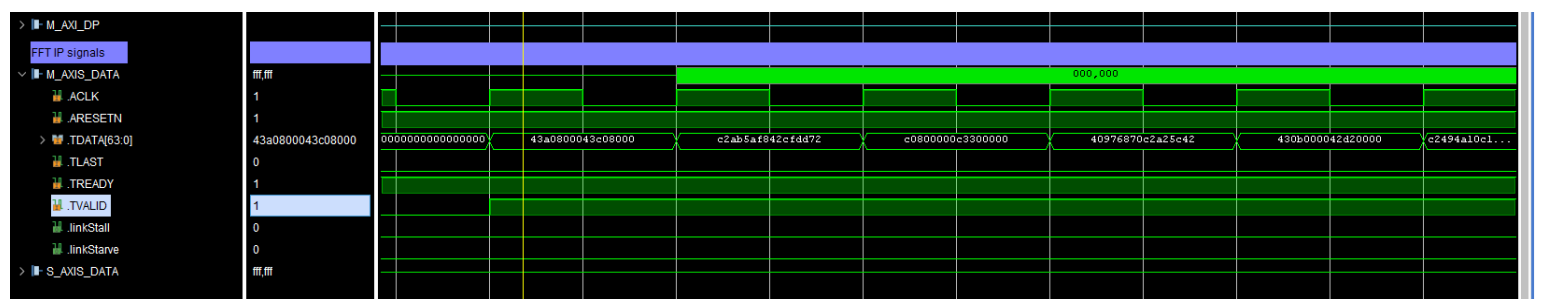


5. FFT Signal Check: To be absolutely sure the math was happening correctly, we performed an FFT signal check. We looked directly at the AXI signals of the FFT IP in the simulation. We checked what exact data signals the FFT IP received at its input and what calculated data it pushed out at its output to verify the hardware was processing the numbers correctly.

FFT IP read data.



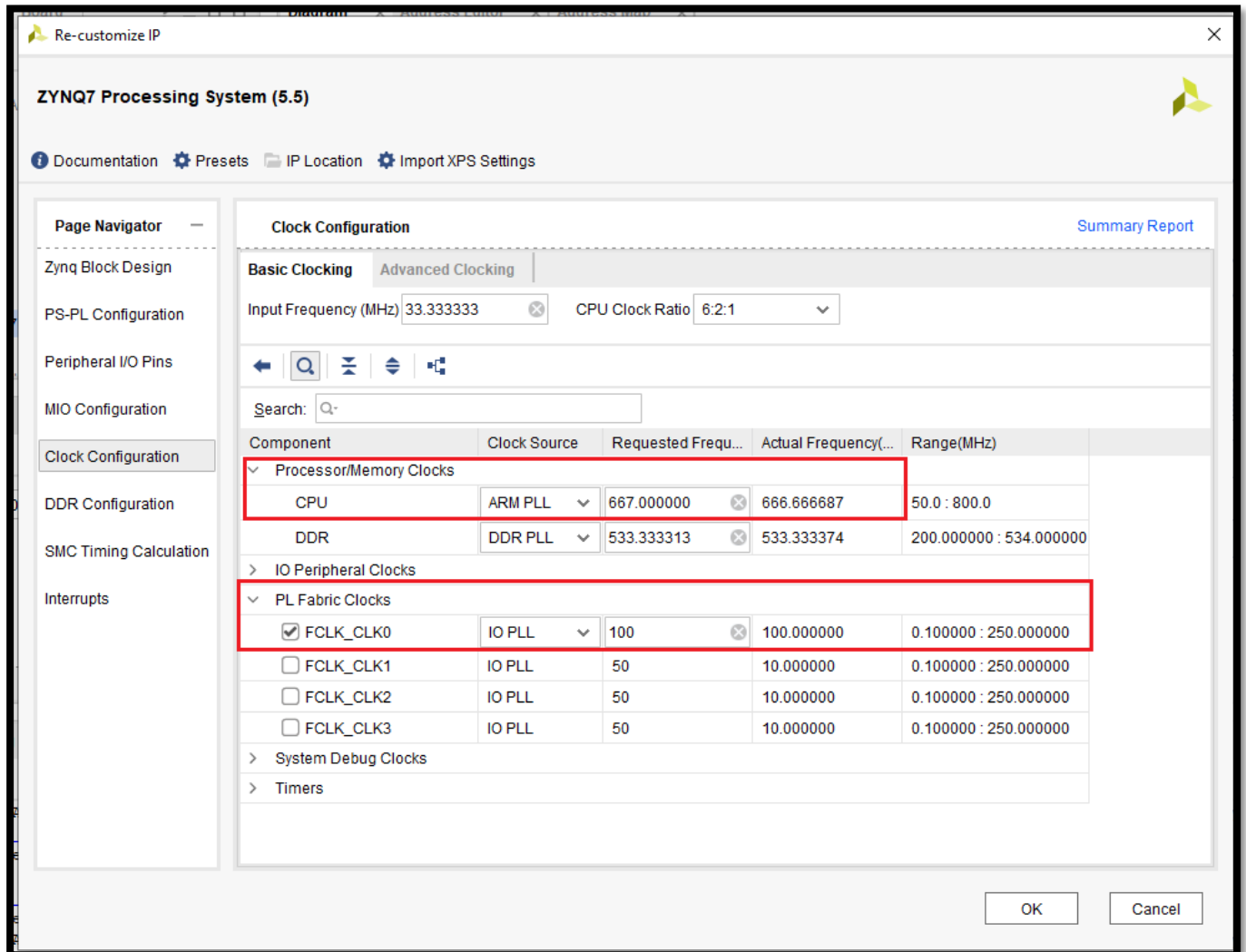
FFT IP write data.



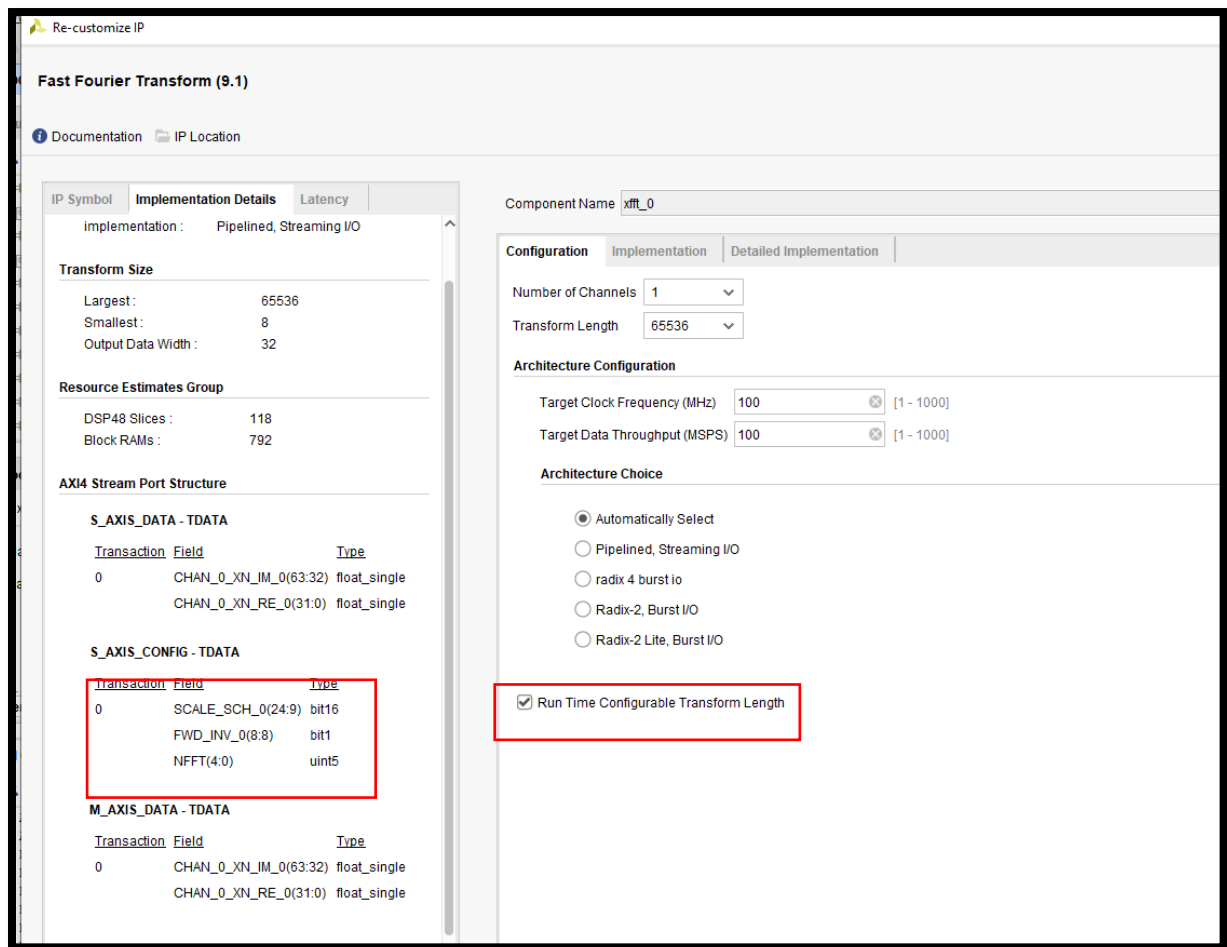
Here, we can confirm that FFT IP has read the correct data and we can check the output data that was stored in the BRAM.

Implementation of FFT IP and DMA IP

with Hardcore Zynq processor



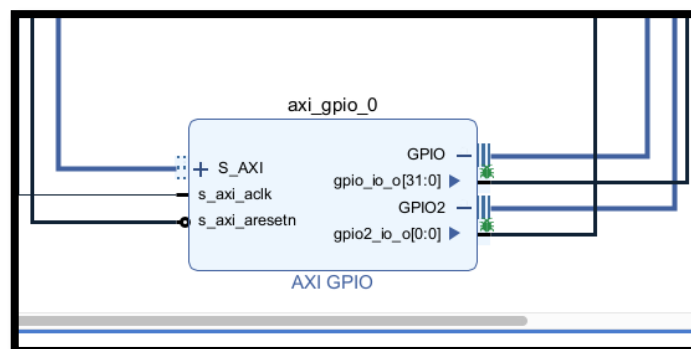
The processor (PS) is operating at 667 MHz and the PL(Programming Logic)(FPGA) is operating at 100 MHz frequency.

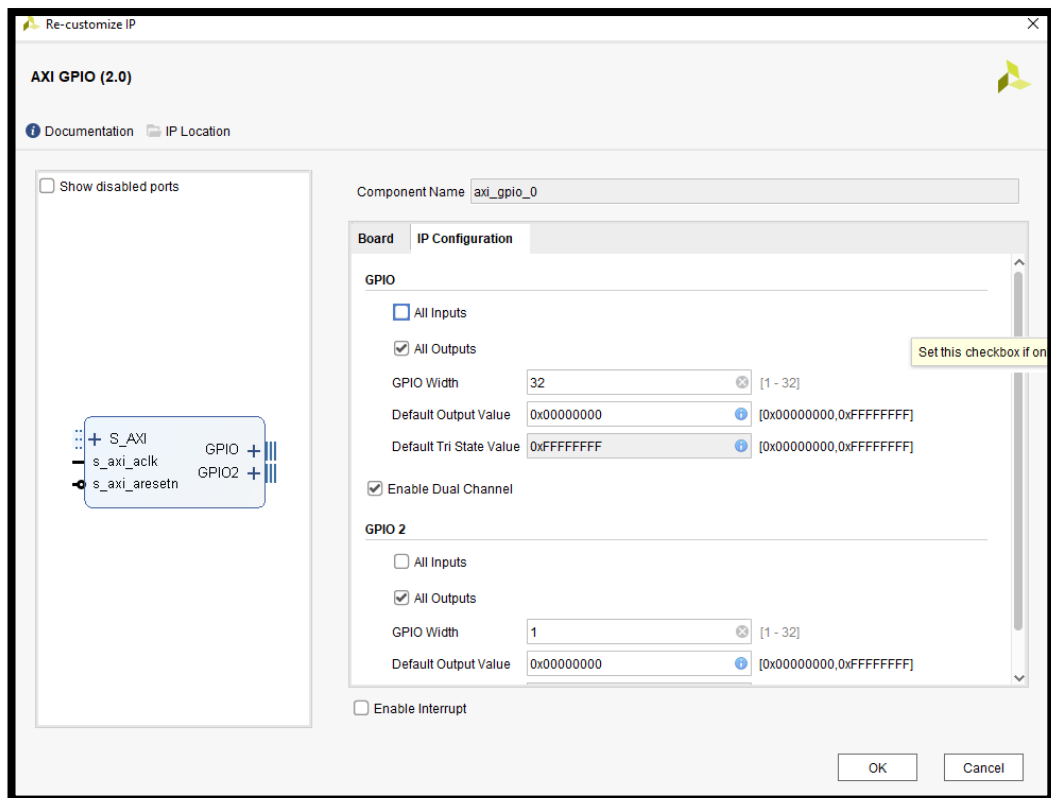


Everything else is same in the FFT IP configuration, and for the DMA IP it's configuration is same as above.

GPIO (General Purpose Input/Output)

GPIO is the simplest way for a processor to talk to custom hardware. We use it like a basic digital switch. The software uses the GPIO to send simple configuration settings (like telling the hardware what FFT size to use) and start signals to the custom logic. Because it is so simple, it is highly reliable.





ILA (Integrated Logic Analyzer)

To check what the signals are doing inside the silicon in real-time, we added an ILA to the design.

ILA is an oscilloscope inside the FPGA chip. It lets us probe the internal wires and watch the signals switch between 0 and 1 live on our computer screen while the board is running.

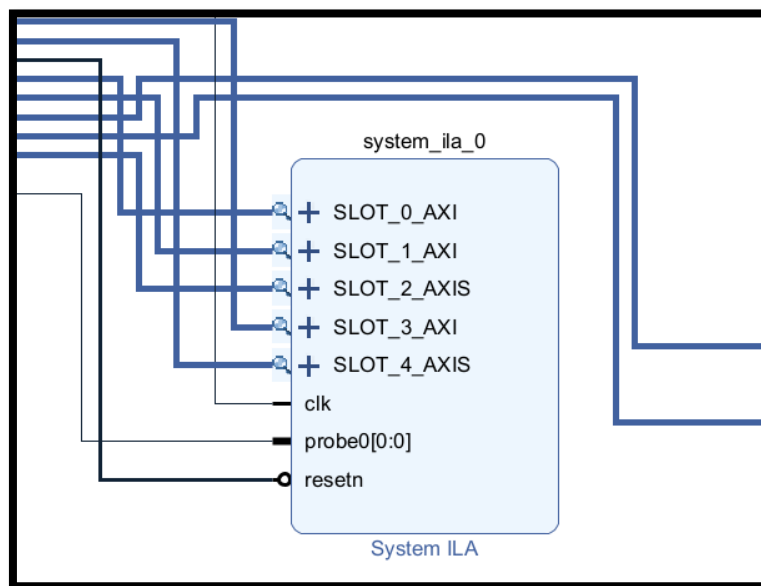
The Disadvantage: The main drawback of an ILA is that it is built using the FPGA's physical resources. It takes up actual logic gates and block RAM. If you put too many probes in your design, the ILA becomes too big and your design might run out of space on the chip.

The Advantages of ILA

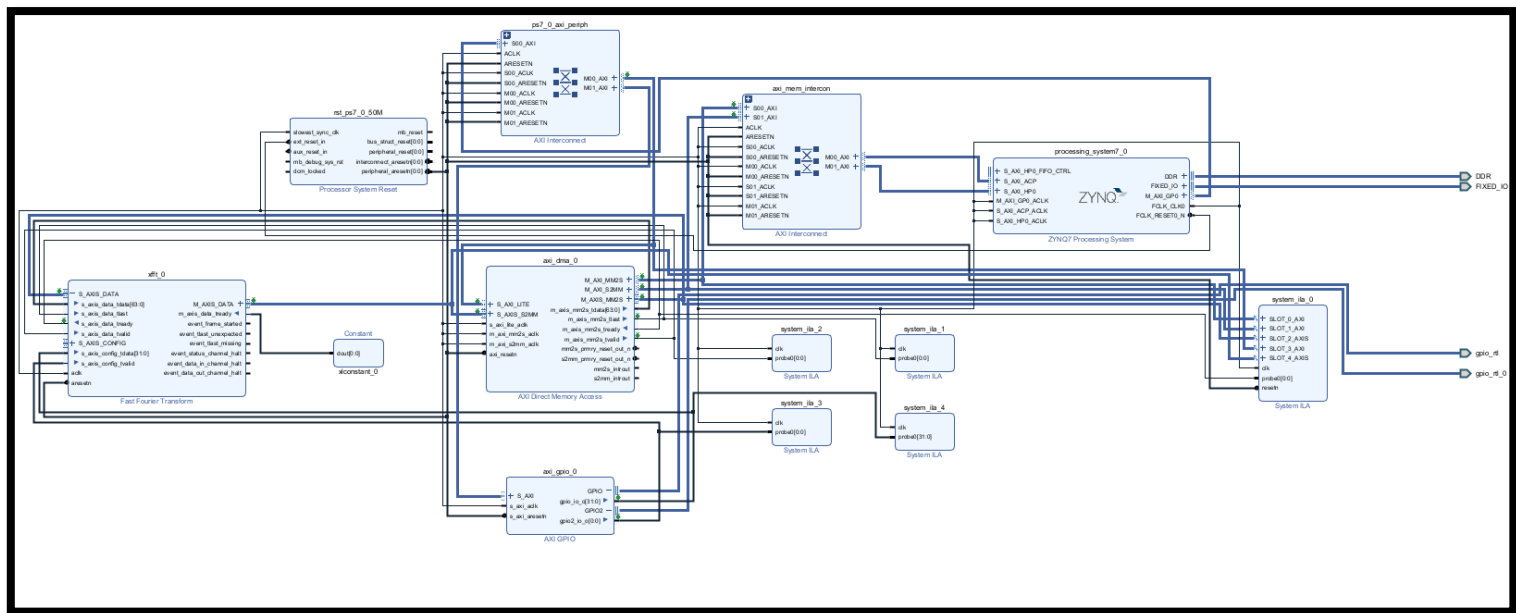
While it takes up space on the chip, the ILA is a lifesaver for finding hard-to-catch bugs. Here is why we use it:

1. **True Real-Time Vision:** Simulation is just a software guess of what *should* happen. The ILA shows you exactly what *is* happening inside the physical silicon while the board is running.

2. **No External Equipment:** You don't need to hook up expensive external oscilloscopes or logic analyzer machines to the pins on your board. The probe is built right into the chip and sends the data directly to your computer over the standard USB programming cable.
3. **Smart Triggering:** You don't have to look through millions of clock cycles of boring data. You can tell the ILA to "wait and only take a snapshot when this specific event happens" (like when a signal goes high at the wrong time).
4. **Catching Hardware Bugs:** It helps you find glitches, memory crashes, and timing issues that only show up on the real physical hardware.



Block Diagram



C Code

Libraries included.

We have the same generate input, input reordering, and FFT function.

```
*helloworld.c  helloworld.c  xaxidma.c  xaxidma_hw.h  xparam.c
34 #include <stdio.h>
35 #include <stdlib.h>
36 #include <complex.h>
37 #include <math.h>
38 #include "xaxidma.h"
39 #include "xgpio.h" // Added for the Configuration GPIO
40 #include "xparameters.h"
41 #include "xtime_l.h"
42 #include "xil_cache.h" // Added for Zynq Cache Management
43
44 #define FFT_Size 128
45
46 #define DMA_IDLE_MASK 0x00000002
47 #define input_sel 1
48 #define fs 10000000.0
49 #define f 1000000.0
50
51 #ifndef M_PI
52 #define M_PI 3.14159265358979323846
53 #endif
54
55
56 void generate_input(float complex* input, int sel, int N) {
57
```

```

140 int main()
141 {
142     int status;
143
144     // --- STEP 1: SET THE FFT SIZE ---
145     u16 NFFT_Value = log2(FFT_Size);    // 5 means 2^5 = 32 points.
146     int N = FFT_Size ;
147
148     // --- STEP 2: CONFIGURE THE HARDWARE FFT VIA GPIO ---
149     XGpio ConfigGpio;
150     status = XGpio_Initialize(&ConfigGpio, XPAR_GPIO_0_DEVICE_ID);
151     if (status != XST_SUCCESS) {
152         // printf("GPIO Init Failed\r\n");
153         return XST_FAILURE;
154     }
155
156     u16 config_word = 0;
157     config_word = config_word | NFFT_Value;    // Put NFFT size into bits 4:0
158     config_word = config_word | (1 << 8);    // Put '1' into bit 8 for Forward FFT
159
160     // Send data to Channel 1 (tdata), lock it in by setting Channel 2 (tvalid) High
161     XGpio_DiscreteWrite(&ConfigGpio, 1, config_word);
162     XGpio_DiscreteWrite(&ConfigGpio, 2, 1);
163     // printf("Hardware FFT reconfigured.\r\n");
164
165     // --- STEP 3: MEMORY ALLOCATION & DATA GEN ---
166     float complex* FFT_input = (float complex*) malloc(N * sizeof(float complex));
167     float complex* FFT_output = (float complex*) malloc(N * sizeof(float complex));
168     float complex* FFT_output_PL = (float complex*) malloc(N * sizeof(float complex));
169
170     float complex* ptr1 = FFT_input ;
171     float complex* ptr2 = FFT_output ;
172     float complex* ptr3 = FFT_output_PL ;
173     generate_input(FFT_input, input_sel, N);
174
175     //--- STEP 4: SOFTWARE (PS) FFT EXECUTION ---
176     XTime time_PS_start, time_PS_end;
177
178     XTime_GetTime(&time_PS_start);
179
180     fft_N(FFT_output, N, FFT_input);
181
182     XTime_GetTime(&time_PS_end);
183

```

First we have configured the GPIO by the FFT size (N), and due to that, the FFT IP will be configured. So, FFT IP is configured according to the GPIO is configured.

```

184
185 float complex ip[FFT_Size] = {0} ;
186 float complex op_PS[FFT_Size] = {0} ;
187 float complex op_PL[FFT_Size] = {0} ;
188 int op_error[FFT_Size] = {0} ;
189
190 // ---STEP 5: INITIALIZE DMA ---
191 XAxiDma_Config *DMA_confptr;
192 XAxiDma AxiDMA;
193
194 DMA_confptr = XAxiDma_LookupConfig(XPAR_AXIDMA_0_DEVICE_ID);
195 status = XAxiDma_CfgInitialize(&AxiDMA, DMA_confptr);
196 if(status != XST_SUCCESS){
197     return XST_FAILURE;
198 }
199
200
201 // ---STEP 6: HARDWARE (PL) FFT EXECUTION ---
202 XTime time_PL_start, time_PL_end;
203 XTime_GetTime(&time_PL_start);
204
205 // Flush Cache BEFORE telling the DMA to look at the DDR Memory
206 Xil_DCacheFlushRange((UINTPTR) FFT_input, (N * sizeof(float complex)));
207 Xil_DCacheFlushRange((UINTPTR) FFT_output_PL, (N * sizeof(float complex)));
208
209 // Start transfers (Setup RX first, then TX)
210 status = XAxiDma_SimpleTransfer(&AxiDMA, (UINTPTR) FFT_output_PL, (N * sizeof(float complex)), XAXIDMA_DEVICE_TO_DMA);
211 status = XAxiDma_SimpleTransfer(&AxiDMA, (UINTPTR) FFT_input, (N * sizeof(float complex)), XAXIDMA_DMA_TO_DEVICE);
212
213 // Wait for TX (MM2S) to finish
214 status = XAxiDma_ReadReg(XPAR_AXI_DMA_0_BASEADDR, 0x4) & XAXIDMA_HALTED_MASK;
215 while(status != 1) {
216     status = XAxiDma_ReadReg(XPAR_AXI_DMA_0_BASEADDR, 0x4) & XAXIDMA_HALTED_MASK;
217 }
218
219 // Wait for RX (S2MM) to finish
220 status = XAxiDma_ReadReg(XPAR_AXI_DMA_0_BASEADDR, 0x34) & XAXIDMA_HALTED_MASK;
221 while(status != 1) {
222     status = XAxiDma_ReadReg(XPAR_AXI_DMA_0_BASEADDR, 0x34) & XAXIDMA_HALTED_MASK;
223 }
224
225 XTime_GetTime(&time_PL_end);
226

```

Initialize the DMA, and transfer the data. And calculate the PL time.

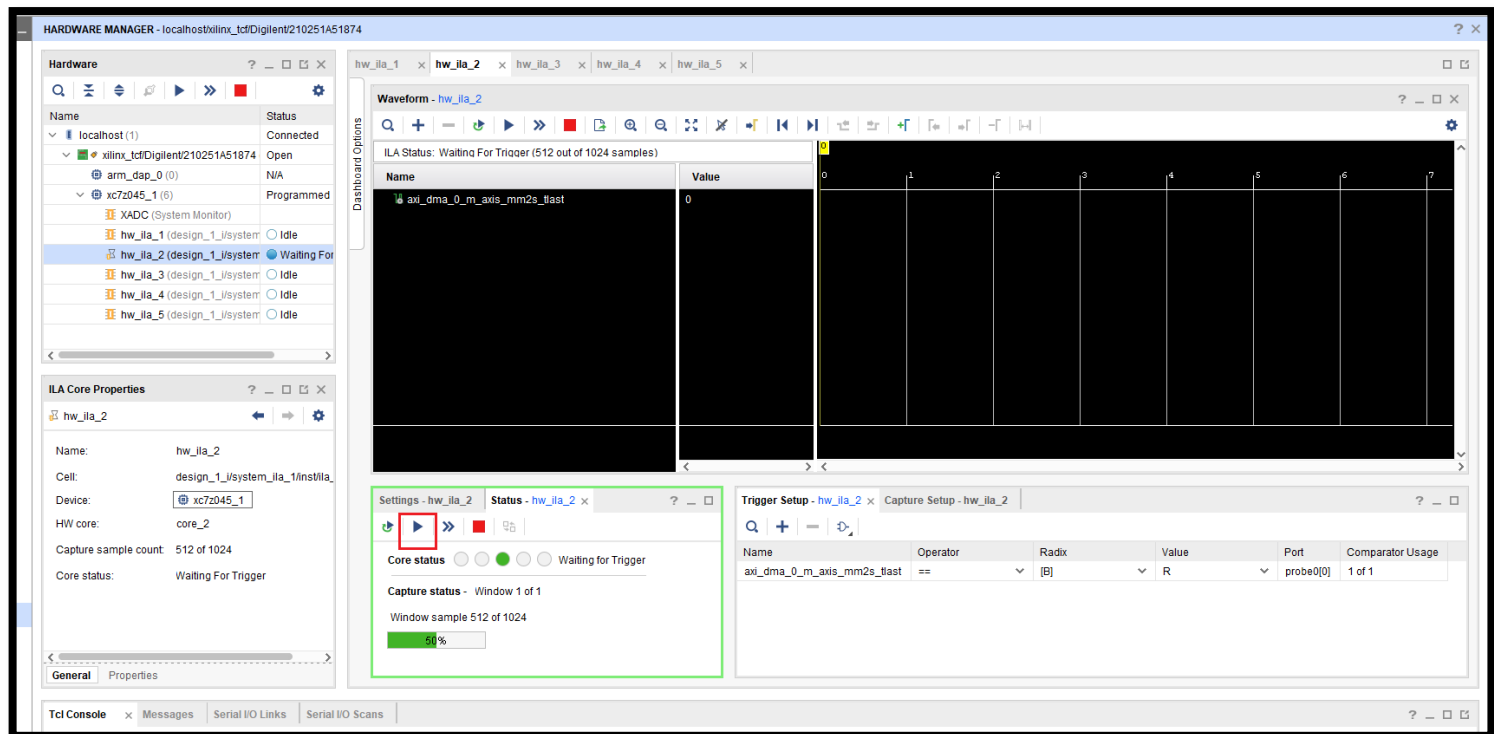
```

230
231 // --- STEP 8: CAL PL FFT Execution time ---
232 for (int i = 0 ; i < N ; i++)
233 {
234
235     ip[i] = *ptr1 ;
236     ptr1++;
237
238     float real2_PL = crealf(*ptr3)*crealf(*ptr3) ;
239     float imag2_PL = cimagf(*ptr3)*cimagf(*ptr3) ;
240     float magnitude_PL = sqrt(real2_PL + imag2_PL) ;
241     op_PL[i] = magnitude_PL ;
242     ptr3 ++ ;
243
244     float real2 = crealf(*ptr2)*crealf(*ptr2) ;
245     float imag2 = cimagf(*ptr2)*cimagf(*ptr2) ;
246     float magnitude = sqrt(real2+imag2) ;
247     op_PS[i] = magnitude ;
248     ptr2 ++ ;
249 }
250
251 int err_flag = 0;
252 int error_cnt = 0 ;int k=0 ;
253 for (int j = 0; j < N; j++) {
254     float diff1 = fabs(crealf(FFT_output[j]) - crealf(FFT_output_PL[j]));
255     float diff2 = fabs(cimagf(FFT_output[j]) - cimagf(FFT_output_PL[j]));
256
257     // Check if error is greater than a tiny floating point tolerance
258     if (diff1 >= 1.0 || diff2 >= 1.0) {
259         err_flag = 1;
260         error_cnt ++ ; //calculate the number of error in the output by
261         op_error[k++] = j ; //comparing PS and PL output.
262                             // also storing the index number of the mismatched output.
263     }
264 }
265
266 float time_ps_us = ((float)1.0 * (time_PS_end - time_PS_start) / COUNTS_PER_SECOND) * 1000000.0;
267 float time_pl_us = ((float)1.0 * (time_PL_end - time_PL_start) / COUNTS_PER_SECOND) * 1000000.0;
268
269 // Clean up
270 free(FFT_input);
271 free(FFT_output);
272 free(FFT_output_PL);
273 return 0;
274 }
275
276

```

Calculate the PS and PL time and compare the output of both the components.

ILA configuration



Open Hardware Manager and then Connect your device (it will auto connect your device when it is powered ON.)

Select the probe you want to capture, and configure its **Trigger Setup**.

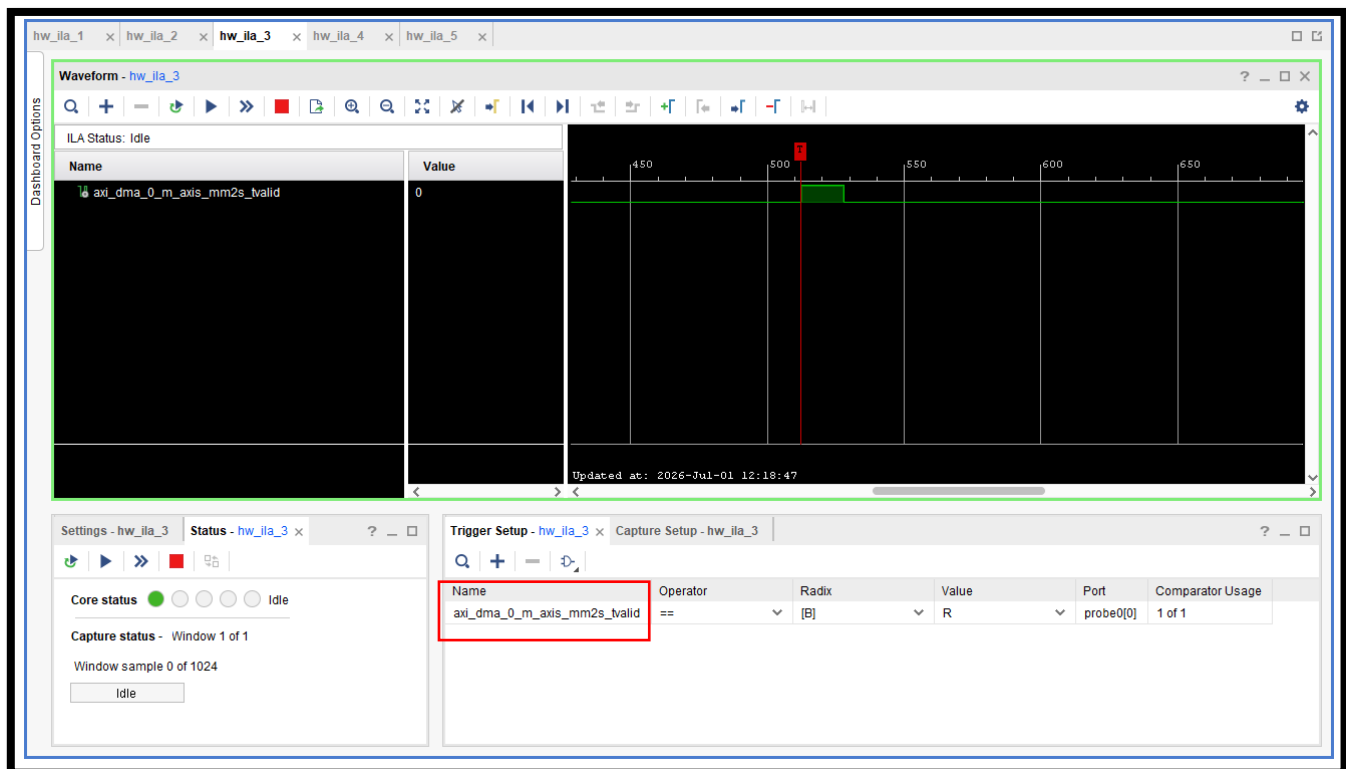
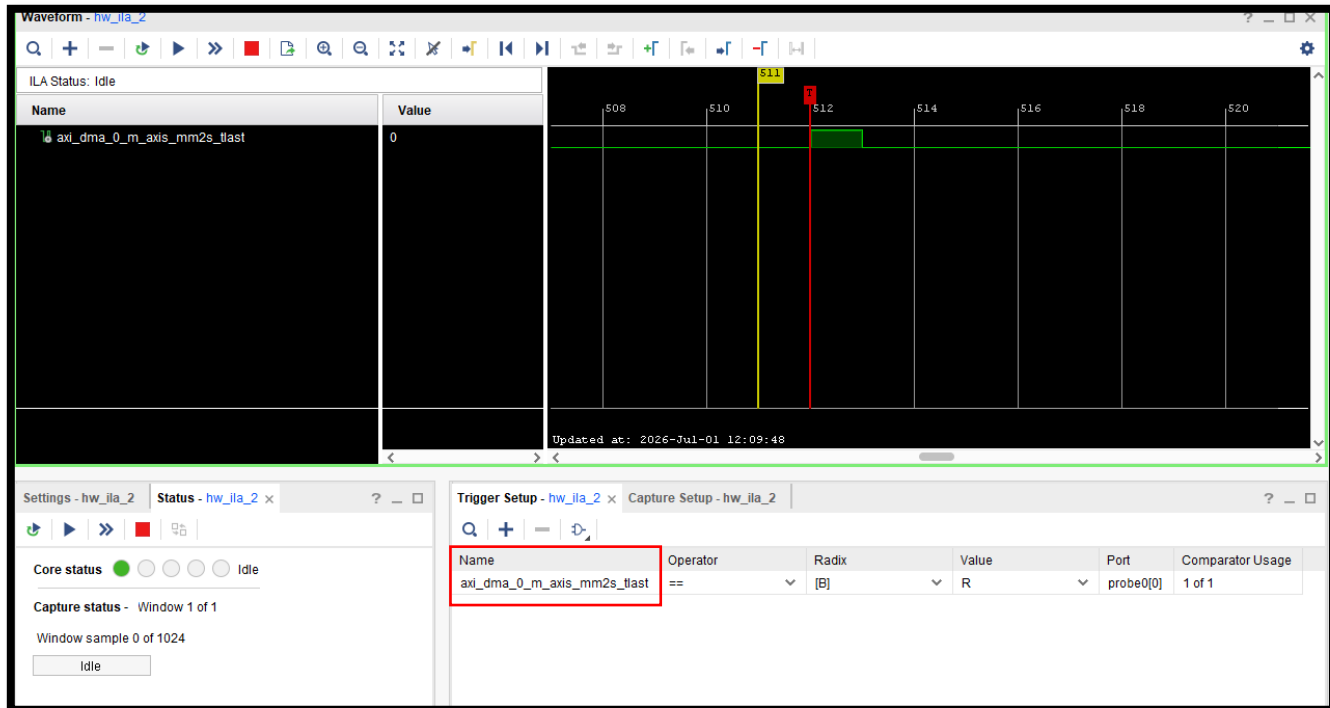
Then program your FPGA.

Then, click on **Run Trigger for this ILA Core**.

After that, Run your whole program, then you can see if the probe has been triggered or not.

Results

After running the program, we can see that ***tlast*** and ***tvalid*** signal has been triggered.



FFT Size :- 8, with NO ERRORS

(x) Variables Breakpoints Expressions Registers		
Name	Type	Value
(x) status	int	0
(x) NFFT_Value	u16	0x0003
(x) N	int	8
> ConfigGpio	XGpio	{BaseAddress=0x41200000, IsReady=0x11111111, InterruptPresent=0, IsDua
(x) config_word	u16	0x0103
> FFT_input	complex float *	0x00110088
> FFT_output	complex float *	0x001100d0
> FFT_output_PL	complex float *	0x00110118
> ptr1	complex float *	0x001100c8
> ptr2	complex float *	0x00110110
> ptr3	complex float *	0x00110158
(x) time_PS_start	XTime	0x00000000ad5498be
(x) time_PS_end	XTime	0x00000000ad54a859
> ip	complex float [8]	[1.000000, 0.9980267+0.06279052i, 0.9921147+0.1253332i, 0.9822872+0.18
> op_PS	complex float [8]	[7.917347, 0.7035575, 0.3632912, 0.2728648, 0.2488126, 0.2658523, 0.34
> op_PL	complex float [8]	[7.917347, 0.7035575, 0.3632912, 0.2728648, 0.2488127, 0.2658523, 0.34
> op_error	int [8]	[0, 0, 0, 0, 0, 0, 0, 0]
> DMA_confptr	XAxiDma_Config *	{DeviceId=0x00000000, BaseAddr=0x40400000, HasStsCntrlStrm=0, HasMm2S=
> AxiDMA	XAxiDma	{RegBase=0x40400000, HasMm2S=1, HasS2Mm=1, Initialized=1, HasSg=0, TxB
(x) time_PL_start	XTime	0x00000000ad54ad37
(x) time_PL_end	XTime	0x00000000ad54b36b
(x) err_flag	int	0
(x) error_cnt	int	0
(x) k	int	0
(x) time_ps_us	float	11.98500
(x) time_pl_us	float	4.763999

FFT Size :- 16, with NO ERRORS

(x) Variables Breakpoints Expressions Registers		
Name	Type	Value
(x) status	int	0
(x) NFFT_Value	u16	0x0004
(x) N	int	16
> ConfigGpio	XGpio	{BaseAddress=0x41200000, IsReady=0x11111111, InterruptPresent=0, IsDua
(x) config_word	u16	0x0104
> FFT_input	complex float *	0x00110088
> FFT_output	complex float *	0x00110110
> FFT_output_PL	complex float *	0x00110198
> ptr1	complex float *	0x00110108
> ptr2	complex float *	0x00110190
> ptr3	complex float *	0x00110218
(x) time_PS_start	XTime	0x000000015ef3027f
(x) time_PS_end	XTime	0x000000015ef323b9
> ip	complex float [16]	[1.000000, 0.9980267+0.06279052i, 0.9921147+0.1253332i, 0.9822872+0.18
> op_PS	complex float [16]	[15.33722, 2.934179, 1.362908, 0.9103795, 0.7037554, 0.5921198, 0.5285
> op_PL	complex float [16]	[15.33722, 2.934179, 1.362908, 0.9103795, 0.7037554, 0.5921198, 0.5285
> op_error	int [16]	[0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
> DMA_confptr	XAxiDma_Config *	{DeviceId=0x00000000, BaseAddr=0x40400000, HasStsCntrlStrm=0, HasMm2S=
> AxiDMA	XAxiDma	{RegBase=0x40400000, HasMm2S=1, HasS2Mm=1, Initialized=1, HasSg=0, TxB
(x) time_PL_start	XTime	0x000000015ef328ad
(x) time_PL_end	XTime	0x000000015ef32f28
(x) err_flag	int	0
(x) error_cnt	int	0
(x) k	int	0
(x) time_ps_us	float	25.51800
(x) time_pl_us	float	4.977000

FFT Size :- 32, with NO ERRORS

Name	Type	Value
(x)- status	int	0
(x)- NFFT_Value	u16	0x0005
(x)- N	int	32
> ConfigGpio	XGpio	{BaseAddress=0x41200000, IsReady=0x11111111, InterruptPresent=0, IsDua
(x)- config_word	u16	0x0105
> FFT_input	complex float *	0x00110088
> FFT_output	complex float *	0x00110190
> FFT_output_PL	complex float *	0x00110298
> ptr1	complex float *	0x00110188
> ptr2	complex float *	0x00110290
> ptr3	complex float *	0x00110398
(x)- time_PS_start	XTime	0x00000000655e38d4
(x)- time_PS_end	XTime	0x00000000655e83a4
> ip	complex float [32]	[1.000000, 0.9980267+0.06279052i, 0.9921147+0.1253332i, 0.9822872+0.18
> op_PS	complex float [32]	[26.88021, 12.65683, 5.142482, 3.246376, 2.388651, 1.903945, 1.595543,
> op_PL	complex float [32]	[26.88021, 12.65683, 5.142482, 3.246376, 2.388651, 1.903945, 1.595543,
> op_error	int [32]	[0, 0,
> DMA_confptr	XAxiDma_Config *	{DeviceId=0x00000000, BaseAddr=0x40400000, HasStsCntrlStrm=0, HasMm2S=
> AxiDMA	XAxiDma	{RegBase=0x40400000, HasMm2S=1, HasS2Mm=1, Initialized=1, HasSg=0, Tx8
(x)- time_PL_start	XTime	0x00000000655e88c6
(x)- time_PL_end	XTime	0x00000000655e90b8
(x)- err_flag	int	0
(x)- error_cnt	int	0
(x)- k	int	0
(x)- time_ps_us	float	57.45600
(x)- time_pl_us	float	6.102000

FFT Size :- 64, with NO ERRORS

Name	Type	Value
(x)- status	int	0
(x)- NFFT_Value	u16	0x0006
(x)- N	int	64
> ConfigGpio	XGpio	{BaseAddress=0x41200000, IsReady=0x11111111, InterruptPresent=0, IsDua
(x)- config_word	u16	0x0106
> FFT_input	complex float *	0x00110088
> FFT_output	complex float *	0x00110290
> FFT_output_PL	complex float *	0x00110498
> ptr1	complex float *	0x00110288
> ptr2	complex float *	0x00110490
> ptr3	complex float *	0x00110698
(x)- time_PS_start	XTime	0x000000006126a9bc
(x)- time_PS_end	XTime	0x000000006127561b
> ip	complex float [64]	[1.000000, 0.9980267+0.06279052i, 0.9921147+0.1253332i, 0.9822872+0.18
> op_PS	complex float [64]	[28.80628, 51.20540, 13.56374, 7.828084, 5.510960, 4.260198, 3.478991,
> op_PL	complex float [64]	[28.80628, 51.20540, 13.56374, 7.828084, 5.510960, 4.260198, 3.478991,
> op_error	int [64]	[0, 0,
> DMA_confptr	XAxiDma_Config *	{DeviceId=0x00000000, BaseAddr=0x40400000, HasStsCntrlStrm=0, HasMm2S=
> AxiDMA	XAxiDma	{RegBase=0x40400000, HasMm2S=1, HasS2Mm=1, Initialized=1, HasSg=0, Tx8
(x)- time_PL_start	XTime	0x0000000061275be2
(x)- time_PL_end	XTime	0x000000006127652e
(x)- err_flag	int	0
(x)- error_cnt	int	0
(x)- k	int	0
(x)- time_ps_us	float	132.3810
(x)- time_pl_us	float	7.140000

FFT Size :- 128, with NO ERRORS

Name	Type	Value
(x)= status	int	0
(x)= NFFT_Value	u16	0x0007
(x)= N	int	128
> ConfigGpio	XGpio	{BaseAddress=0x41200000, IsReady=0x11111111, InterruptPresent=0, IsDua
(x)= config_word	u16	0x0107
> FFT_input	complex float *	0x00110088
> FFT_output	complex float *	0x00110490
> FFT_output_PL	complex float *	0x00110898
> ptr1	complex float *	0x00110488
> ptr2	complex float *	0x00110890
> ptr3	complex float *	0x00110c98
(x)= time_PS_start	XTime	0x00000000ad3030
(x)= time_PS_end	XTime	0x000000008ae0baf6
> ip	complex float [128]	[1.000000, 0.9980267+0.06279052i, 0.9921147+0.1253332i, 0.9822872+0.18
> op_PS	complex float [128]	[24.53023, 112.1206, 43.60440, 18.25748, 11.55032, 8.450861, 6.666073,
> op_PL	complex float [128]	[24.53023, 112.1206, 43.60441, 18.25748, 11.55032, 8.450859, 6.666073,
> op_error	int [128]	[0, 0,
> DMA_confptr	XAxiDma_Config *	{DeviceId=0x00000000, BaseAddr=0x40400000, HasStsCntrlStrm=0, HasMm2S=
> AxiDMA	XAxiDma	{RegBase=0x40400000, HasMm2S=1, HasS2Mm=1, Initialized=1, HasSg=0, TxB
(x)= time_PL_start	XTime	0x000000008ae0c1ac
(x)= time_PL_end	XTime	0x000000008ae0cddf
(x)= err_flag	int	0
(x)= error_cnt	int	0
(x)= k	int	0
(x)= time_ps_us	float	303.1860
(x)= time_pl_us	float	9.368999

FFT Size :- 256, with 15 ERRORS

Name	Type	Value
(x)= status	int	0
(x)= NFFT_Value	u16	0x0008
(x)= N	int	256
> ConfigGpio	XGpio	{BaseAddress=0x41200000, IsReady=0x11111111, InterruptPresent=0, IsDua
(x)= config_word	u16	0x0108
> FFT_input	complex float *	0x00110088
> FFT_output	complex float *	0x00110890
> FFT_output_PL	complex float *	0x00111098
> ptr1	complex float *	0x00110888
> ptr2	complex float *	0x00111090
> ptr3	complex float *	0x00111898
(x)= time_PS_start	XTime	0x0000000050a04a81
(x)= time_PS_end	XTime	0x0000000050a3d27c
> ip	complex float [256]	[1.000000, 0.9980267+0.06279052i, 0.9921147+0.1253332i, 0.9822872+0.18
> op_PS	complex float [256]	[31.27233, 51.31336, 142.9367, 181.9190, 55.58898, 32.80980, 23.27551,
> op_PL	complex float [256]	[31.27232, 51.31337, 142.9367, 181.9190, 55.58898, 32.80979, 23.27551,
> op_error	int [256]	[241, 242, 243, 244, 245, 246, 247, 248, 249, 250, 251, 252, 253, 254,
> DMA_confptr	XAxiDma_Config *	{DeviceId=0x00000000, BaseAddr=0x40400000, HasStsCntrlStrm=0, HasMm2S=
> AxiDMA	XAxiDma	{RegBase=0x40400000, HasMm2S=1, HasS2Mm=1, Initialized=1, HasSg=0, TxB
(x)= time_PL_start	XTime	0x0000000050a3db05
(x)= time_PL_end	XTime	0x0000000050a3edd5
(x)= err_flag	int	1
(x)= error_cnt	int	15
(x)= k	int	15
(x)= time_ps_us	float	694.2570
(x)= time_pl_us	float	14.44800

FFT Size :- 512, with 16 ERRORS

(x)= Variables Breakpoints Expressions Registers		
Name	Type	Value
(x)= status	int	0
(x)= NFFT_Value	u16	0x0009
(x)= N	int	512
> ConfigGpio	XGpio	{BaseAddress=0x41200000, IsReady=0x11111111, InterruptPresent=0, IsDua
(x)= config_word	u16	0x0109
> FFT_input	complex float *	0x00110088
> FFT_output	complex float *	0x00111090
> FFT_output_PL	complex float *	0x00112098
> ptr1	complex float *	0x00111088
> ptr2	complex float *	0x00112090
> ptr3	complex float *	0x00113098
(x)= time_PS_start	XTime	0x000000005ed58e4a
(x)= time_PS_end	XTime	0x000000005edd872f
> ip	complex float [512]	[1.000000, 0.9980267+0.06279052i, 0.9921147+0.1253332i, 0.9822872+0.18
> op_PS	complex float [512]	[11.71972, 14.56346, 19.23034, 28.30034, 53.56741, 499.9581, 68.17651,
> op_PL	complex float [512]	[11.71972, 14.56346, 19.23034, 28.30035, 53.56741, 499.9582, 68.17652,
> op_error	int [512]	[496, 497, 498, 499, 500, 501, 502, 503, 504, 505, 506, 507, 508, 509,
> DMA_confptr	XAxiDma_Config *	{DeviceId=0x00000000, BaseAddr=0x40400000, HasStsCntrlStrm=0, HasMm2S=
> AxiDMA	XAxiDma	{RegBase=0x40400000, HasMm2S=1, HasS2Mm=1, Initialized=1, HasSg=0, TxB
(x)= time_PL_start	XTime	0x000000005edd93ce
(x)= time_PL_end	XTime	0x000000005eddb2b4
(x)= err_flag	int	1
(x)= error_cnt	int	16
(x)= k	int	16
(x)= time_ps_us	float	1567.407
(x)= time_pl_us	float	23.73000

FFT Size :- 1024, with 24 ERRORS

(x)= Variables Breakpoints Expressions Registers		
Name	Type	Value
(x)= status	int	0
(x)= NFFT_Value	u16	0x000a
(x)= N	int	1024
> ConfigGpio	XGpio	{BaseAddress=0x41200000, IsReady=0x11111111, InterruptPresent=0, IsDua
(x)= config_word	u16	0x010a
> FFT_input	complex float *	0x00110088
> FFT_output	complex float *	0x00112090
> FFT_output_PL	complex float *	0x00114098
> ptr1	complex float *	0x00112088
> ptr2	complex float *	0x00114090
> ptr3	complex float *	0x00116098
(x)= time_PS_start	XTime	0x0000000078fd494e
(x)= time_PS_end	XTime	0x00000000790f0dfc
> ip	complex float [1024]	[1.000000, 0.9980267+0.06279052i, 0.9921147+0.1253332i, 0.9822872+0.18
> op_PS	complex float [1024]	[21.79341, 24.15128, 27.08149, 30.82130, 35.75983, 42.58353, 52.62600,
> op_PL	complex float [1024]	[21.79341, 24.15128, 27.08148, 30.82130, 35.75982, 42.58353, 52.62601,
> op_error	int [1024]	[1000, 1001, 1002, 1003, 1004, 1005, 1006, 1007, 1008, 1009, 1010, 101
> DMA_confptr	XAxiDma_Config *	{DeviceId=0x00000000, BaseAddr=0x40400000, HasStsCntrlStrm=0, HasMm2S=
> AxiDMA	XAxiDma	{RegBase=0x40400000, HasMm2S=1, HasS2Mm=1, Initialized=1, HasSg=0, TxB
(x)= time_PL_start	XTime	0x00000000790f227c
(x)= time_PL_end	XTime	0x00000000790f589b
(x)= err_flag	int	1
(x)= error_cnt	int	24
(x)= k	int	24
(x)= time_ps_us	float	3493.386
(x)= time_pl_us	float	41.56500

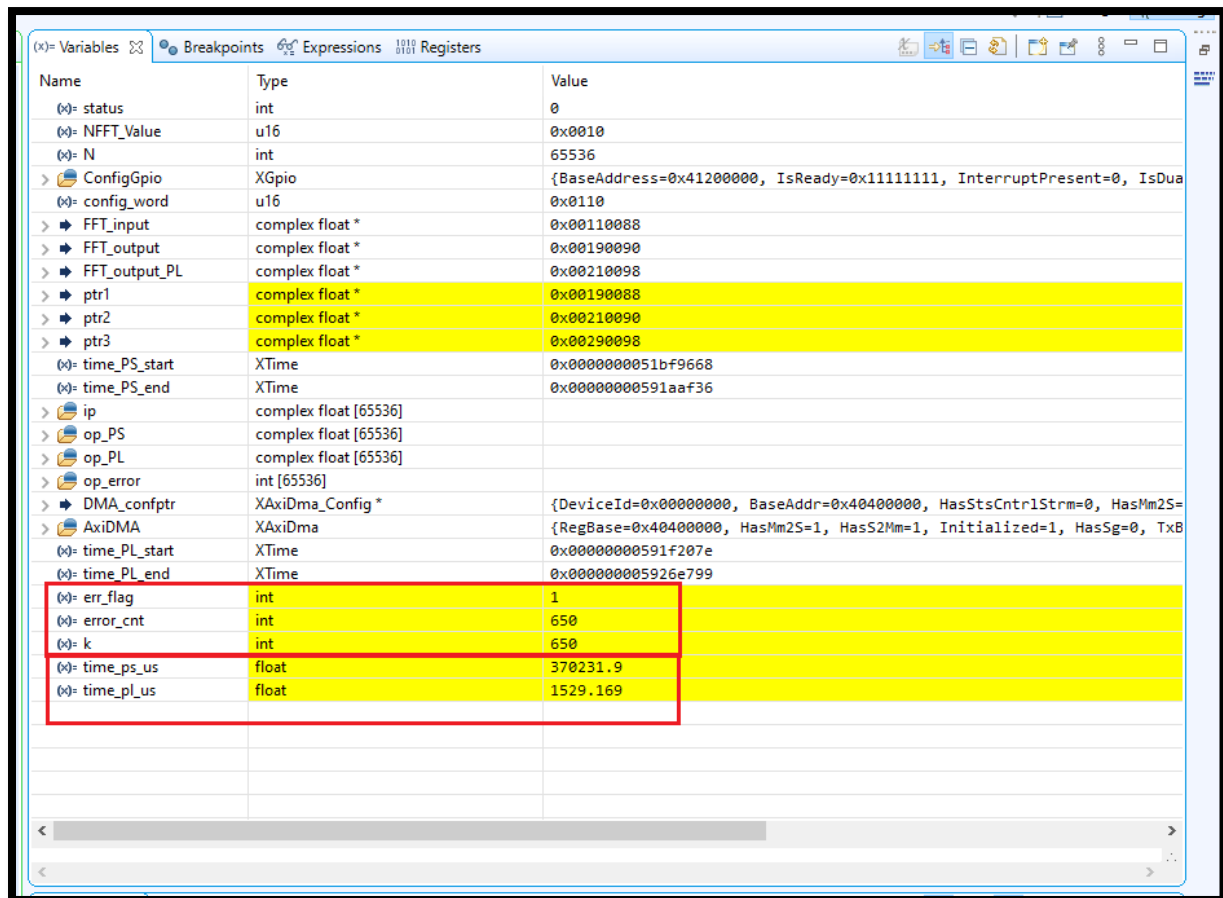
FFT Size :- 2048, with 45 ERRORS

(x)= Variables Breakpoints Expressions Registers		
Name	Type	Value
(x)= status	int	0
(x)= NFFT_Value	u16	0x000b
(x)= N	int	2048
> ConfigGpio	XGpio	{BaseAddress=0x41200000, IsReady=0x11111111, InterruptPresent=0, IsDua
(x)= config_word	u16	0x010b
> FFT_input	complex float *	0x00110088
> FFT_output	complex float *	0x00114090
> FFT_output_PL	complex float *	0x00118098
> ptr1	complex float *	0x00114088
> ptr2	complex float *	0x00118090
> ptr3	complex float *	0x0011c098
(x)= time_PS_start	XTime	0x0000000068cc2020
(x)= time_PS_end	XTime	0x0000000068f34d26
> ip	complex float [2048]	[1.000000, 0.9980267+0.06279052i, 0.9921147+0.1253332i, 0.9822872+0.18
> op_PS	complex float [2048]	[31.77336, 33.40403, 35.21104, 37.22485, 39.48318, 42.03321, 44.93550,
> op_PL	complex float [2048]	[31.77335, 33.40403, 35.21103, 37.22485, 39.48317, 42.03322, 44.93550,
> op_error	int [2048]	[2003, 2004, 2005, 2006, 2007, 2008, 2009, 2010, 2011, 2012, 2013, 201
> DMA_confptr	XAxiDma_Config *	{DeviceId=0x00000000, BaseAddr=0x40400000, HasStsCntrlStrm=0, HasMm2S=
> AxiDMA	XAxiDma	{RegBase=0x40400000, HasMm2S=1, HasS2Mm=1, Initialized=1, HasSg=0, Tx8
(x)= time_PL_start	XTime	0x0000000068f3718d
(x)= time_PL_end	XTime	0x0000000068f3d769
(x)= err_flag	int	1
(x)= error_cnt	int	45
(x)= k	int	45
(x)= time_ps_us	float	7702.290
(x)= time_pl_us	float	78.22800

FFT Size :- 4096, with 6 ERRORS

(x)= Variables Breakpoints Expressions Registers		
Name	Type	Value
(x)= status	int	0
(x)= NFFT_Value	u16	0x000c
(x)= N	int	4096
> ConfigGpio	XGpio	{BaseAddress=0x41200000, IsReady=0x11111111, InterruptPresent=0, IsDua
(x)= config_word	u16	0x010c
> FFT_input	complex float *	0x00110088
> FFT_output	complex float *	0x00118090
> FFT_output_PL	complex float *	0x00120098
> ptr1	complex float *	0x00118088
> ptr2	complex float *	0x00120090
> ptr3	complex float *	0x00128098
(x)= time_PS_start	XTime	0x000000004b8b9cb6
(x)= time_PS_end	XTime	0x000000004be1831d
> ip	complex float [4096]	[1.000000, 0.9980267+0.06279052i, 0.9921147+0.1253332i, 0.9822872+0.18
> op_PS	complex float [4096]	[3.990054, 4.090152, 4.193758, 4.306534, 4.426394, 4.546388, 4.673588,
> op_PL	complex float [4096]	[3.990054, 4.090157, 4.193771, 4.306541, 4.426398, 4.546386, 4.673592,
> op_error	int [4096]	[4090, 4091, 4092, 4093, 4094, 4095, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
> DMA_confptr	XAxiDma_Config *	{DeviceId=0x00000000, BaseAddr=0x40400000, HasStsCntrlStrm=0, HasMm2S=
> AxiDMA	XAxiDma	{RegBase=0x40400000, HasMm2S=1, HasS2Mm=1, Initialized=1, HasSg=0, Tx8
(x)= time_PL_start	XTime	0x000000004be1c6fe
(x)= time_PL_end	XTime	0x000000004be28cdf
(x)= err_flag	int	1
(x)= error_cnt	int	6
(x)= k	int	6
(x)= time_ps_us	float	16888.63
(x)= time_pl_us	float	151.9710

FFT Size :- 65536, with 650 ERRORS



Name	Type	Value
(x)- status	int	0
(x)- NFFT_Value	u16	0x0010
(x)- N	int	65536
> ConfigGpio	XGpio	{BaseAddress=0x41200000, IsReady=0x11111111, InterruptPresent=0, IsDua
(x)- config_word	u16	0x0110
> FFT_input	complex float *	0x00110088
> FFT_output	complex float *	0x00190090
> FFT_output_PL	complex float *	0x00210098
> ptr1	complex float *	0x00190088
> ptr2	complex float *	0x00210090
> ptr3	complex float *	0x00290098
(x)- time_PS_start	XTime	0x0000000051bf9668
(x)- time_PS_end	XTime	0x00000000591aaf36
> ip	complex float [65536]	
> op_PS	complex float [65536]	
> op_PL	complex float [65536]	
> op_error	int [65536]	
> DMA_confptr	XAxiDma_Config *	{DeviceId=0x00000000, BaseAddr=0x40400000, HasStsCntrlStrm=0, HasMm2S=
> AxiDMA	XAxiDma	{RegBase=0x40400000, HasMm2S=1, HasS2Mm=1, Initialized=1, HasSg=0, TxB
(x)- time_PL_start	XTime	0x00000000591f207e
(x)- time_PL_end	XTime	0x000000005926e799
(x)- err_flag	int	1
(x)- error_cnt	int	650
(x)- k	int	650
(x)- time_ps_us	float	370231.9
(x)- time_pl_us	float	1529.169

The error count increases when the difference between the PS output and PL output is greater than 1.0

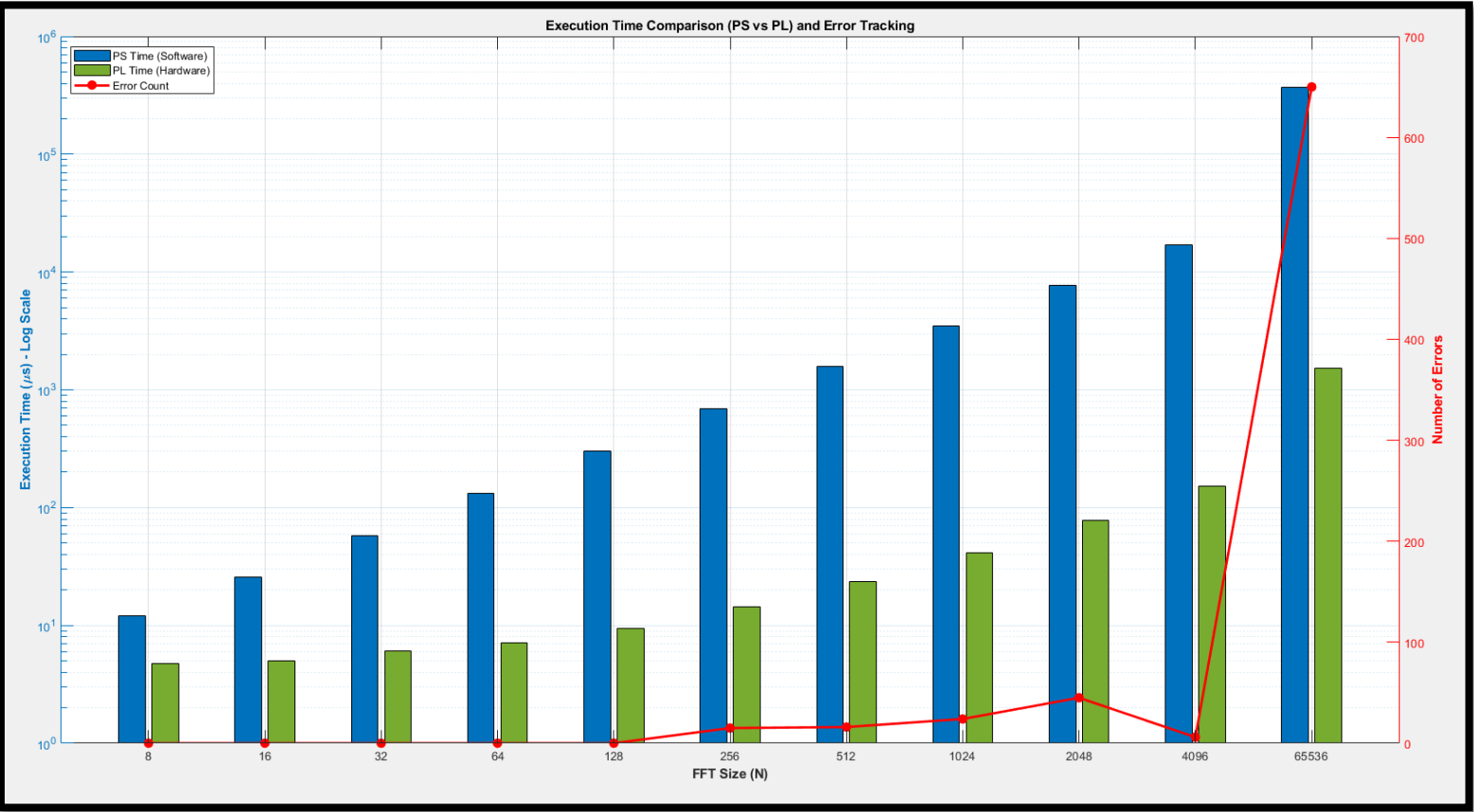
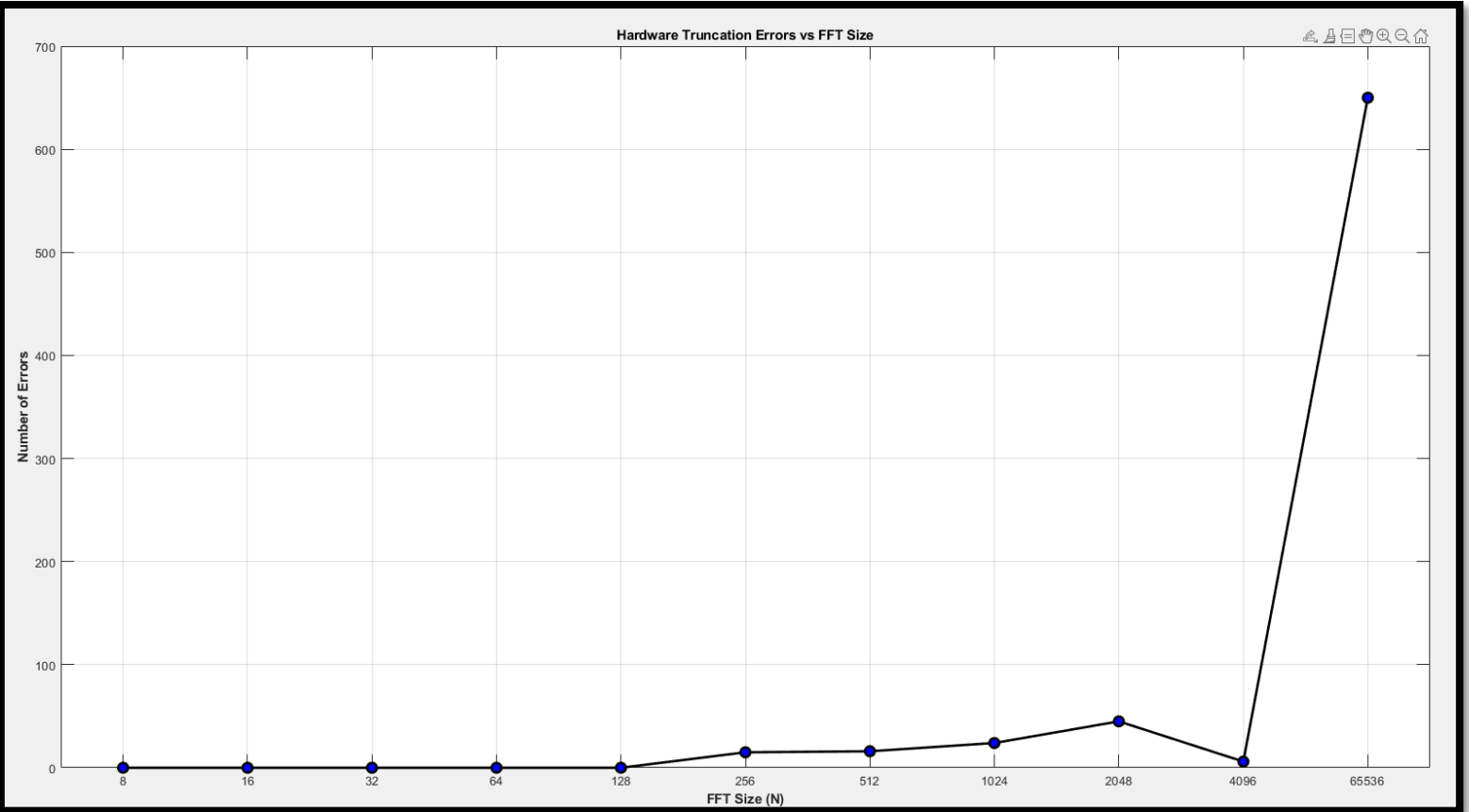
And the error flag will be 0 only when there is NO error, i.e. when the difference between the PS output and PL is less than or equal to 1.0

Execution Time of FFT with DMA IP and FFT IP implemented.

FFT Size	Time (in us) (in microseconds)			Speed Increase Factor	Error Count
	<u>PS Time (us)</u>	<u>Observation 1 PL Time</u>	<u>Observation 2 PL Time</u>	<u>APPROX</u>	
8	11.98500	4.763999	4.763989	2.52	0
16	25.51800	4.977000	4.977000	5.13	0
32	57.45600	6.102000	6.101000	9.42	0
64	132.38100	7.140000	7.140000	18.54	0
128	303.18600	9.368999	9.368799	32.36	0
256	694.25700	14.448000	14.449000	48.05	15
512	1 567.40700	23.730000	23.730000	66.05	16
1024	3 493.38600	41.565000	41.585000	84.03	24
2048	7 702.29000	78.228000	78.228000	98.46	45
4096	16 888.63000	151.971000	151.972000	111.13	6
65536	370 231.90000	1 529.169000	1 530.169000	242.03	650

Speed Increase Factor = (PS Time) / (Mean PL Time)

Mean PL Time = (Observation 1 + Observation 2) / 2



CONCLUSION

The implementation of the FFT and DMA IPs was successfully carried out across two distinct processor environments.

- **MicroBlaze (Softcore) for Simulation and Verification:**
We kept the implementation simple using a fixed FFT transform length of $N = 8$ and hardcoded hexadecimal input values, the system could be easily observed in a software simulation environment. This allowed for clear, step-by-step verification of the AXI signals (tvalid, tlast, etc.) as data moved from the MicroBlaze, into the BRAM, through the DMA, and into the FFT IP.
- **Zynq (Hardcore) for Scalability and Performance:** The Zynq processor was utilized to test the design's physical performance and scalability. Operating at a 667 MHz and Programmable Logic at 100 MHz, the Zynq's ARM processor handled advanced dynamic configurations. Rather than a fixed size, the Zynq implementation used GPIO to **dynamically reconfigure the FFT size** at run-time (up to 65,536 points).
- **Hardware vs. Software:** Unlike the MicroBlaze simulation, the Zynq implementation utilized an Integrated Logic Analyzer (ILA) to probe the physical silicon signals in real-time. Furthermore, the Zynq system allowed for direct benchmarking between the software execution (PS) and the hardware execution (PL).

The results conclusively proved the efficiency of the PL, showing that while minor truncation errors occur at larger sizes, the hardware execution scales impressively, running up to **242.03 times faster** than the software for a 65,536-point FFT.